OmniCode: Chapter 6: Track 1 - Runtime Environment & Evaluator

SVO Verbs, Causal Clock, Transactions & Rollback

CHAPTER IDENTIFIER
CH 06

NOTEBOOKLM WORD COUNT
15,294 words

CATALOGED CHAPTER FILES
19 files (3,761 lines)

GENERATION DATE
2026-10-16 20:35:55

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How does runtime guarantee forward time in rollback?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (19 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/README.md**	MD	109	761	6.1
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/connectives.rs**	RS	160	599	7.1
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/core.rs**	RS	728	2,384	32.9
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/mod.rs**	RS	60	375	3.0
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/relations.rs**	RS	313	1,183	15.0
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/vectors.rs**	RS	254	929	12.7
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/verbs.rs**	RS	214	859	10.0
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/universal/decoder.rs**	RS	280	1,156	10.0
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/universal/mod.rs**	RS	38	171	1.7
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/README.md**	MD	160	1,012	8.3
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/mod.rs**	RS	76	393	3.4
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/hardware/mod.rs**	RS	39	175	1.7
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/hardware/registers.rs**	RS	166	798	7.3
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/environment.rs**	RS	291	1,098	12.2
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/frame.rs**	RS	95	421	3.7
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/mod.rs**	RS	39	180	1.7
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/snapshot.rs**	RS	109	423	4.1
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/universal/mod.rs**	RS	34	167	1.6
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/universal/values.rs**	RS	596	2,210	23.6

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/README.md MD 109 LINES 761 WORDS 6.1 KB

```
# OmniCode Runtime Evaluator Subsystem

```text
================================================================================
BLOCK 1: METADATA & ARCHITECTURAL MANIFESTO
================================================================================
Subsystem: OmniCode Host Engine & Virtual Machine Evaluator
Layer:     AST Execution Engine & Quad-State Operational Reducer
Path:      <TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/
================================================================================
```

> **"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."**
> — *Exodus 31:18 (KJV)*
> *(The immutable inscription of atomic runes onto bounded physical tablets).*
> **"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."**
> — *Proverbs 24:27 (KJV)*
> *(The discipline of proving atomic sub-engines and verification invariants before constructing complex systems).*
> **"Remove not the ancient landmark, which thy fathers have set."**
> — *Proverbs 22:28 (KJV)*
> *(The preservation of historical memory boundaries, silicon addresses, and machine constants).*

---

1. Architectural Mission

The **Evaluator Subsystem** is the operational execution engine of the OmniCode Virtual Machine. It reduces Abstract Syntax Trees (AST) under balanced ternary truth logic (`+1:PROCEED`, `0:AWAIT`, `-1:ROLLBACK`, `NULL:HALT`), evaluating causal vectors, relational algebra, and Subject-Verb-Object (SVO) action verbs.

By decoupling the Evaluator into specialized sub-modules:

1. **Relational Algebra (`relations.rs`)** operates independently of causal vectors, leveraging universal scalar unwrappers and cross-substrate glyph equality from the Environment.
2. **Directional Causality (`vectors.rs`)** guarantees non-corrupting state propagation without side-effects leaking back into source expressions.
3. **Action Verbs (`verbs.rs`)** drive the atomic Character rune lifecycle (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`) with mathematical fidelity.
4. **Adversative Caveats (`connectives.rs`)** enforce reversible computing under `! BUT` by rolling back state transactions when guard conditions fail.

---

2. Topological Layer Structure

```bash
src/runtime/evaluator/
├── universal/                 <-- Tier 1: Universal AST Decoders & Trit Transformers
│   ├── decoder.rs             <-- extract_symbol_name, extract_trits, trits_to_integer
│   └── mod.rs                 <-- Re-exports Universal Decoders
│
├── verbs.rs                   <-- Tier 2: SVO Action Verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE)
├── relations.rs               <-- Tier 2: Being Relations (=, !=, ~=, <=>, <=, >=) & Capacity Bounds
├── vectors.rs                 <-- Tier 2: Directional Causal Propagation Vectors (--> , <--, ^--, v--)
├── connectives.rs             <-- Tier 2: Logical Connectives (! BUT, & AND, | OR, -- NOT, ::)
│
├── core.rs                    <-- Tier 3: Evaluator Coordinator, AST Dispatch Loop & Module Loader
├── mod.rs                     <-- Subsystem Facade & Backward-Compatible API
└── README.md                  <-- Architectural Manifesto & Specification
```

---

3. Subsystem Breakdown

Universal (`universal/decoder.rs`)

- **Symbol Extraction**: Normalizes heterogeneous AST representations (`Identifier`, `Polarity`, `[ ROLL 1 ]`, `[ Verify ]`) into canonical symbol strings.
- **Trit Extraction & Horner Expansion**: Converts 5-trit balanced ternary representations to integer values using Horner's polynomial method ($\sum_{i=0}^{N-1} t_i \cdot 3^i$).
- **Mathematical Invariant**: Guarantees byte-to-trit parity (e.g. $65 \iff [-1, 1, 1, -1, 1]$).
```

```
### SVO Action Verbs (`verbs.rs`)

- **`INSCRIBE`**: Outside-in imposition of glyph bytes into 5-trit Cell containers.
- **`TRANSDUCE`**: Bijective Radix-3 conversion of 8-bit bytes to 5 balanced trits.
- **`PROJECT`**: Outward projection of Character runes to hardware Target buffers (`%into` port / 0x000B8000).
- **`SEQUENCE`**: Continuous accumulation of Character runes into the text ribbon.
- **Vessel Operations**: Memory vessel indexed access and dynamic Storehouse partitioning.

### Relational Algebra (`relations.rs`)

- Evaluates Being Relations under balanced ternary QuadState truth.
- Distinguishes **Observational Relations** (e.g. `{ Character' = Glyph }` checking equality via `is_glyph_equal`) from **Imperative State Advances** (e.g. `[ Status' = +1:PROCEED ]`).
- Enforces invariant capacity bound checks via `capacity_in_cells()` (e.g. `{ Character' <= [ 1 Cell ] } \implies +1:PROCEED`).

### Directional Causality (`vectors.rs`)

- **`-->` (ActUpon)**: Propels causal action forward without mutating or corrupting the source expression.
- **`<--` (ActAgain)**: Manages retroaction, feedback loops, and memory reads.
- **`^--` (BuildsTo)**: Hierarchical composition.
- **`v--` (BreaksDownFrom)**: Storehouse and vessel memory partitioning.

### Logical Connectives (`connectives.rs`)

- **`! BUT`**: Adversative caveat gate wrapping actions in atomic transactions; unwinds with `rollback_transaction()` on `-1:ROLLBACK`.
- **`& AND`, `| OR`, `-- NOT`**: Balanced ternary truth logic.
- **`:: IN_RELATION_TO`**: Module imports (`import :: lib.core.memory`) and namespace path resolution.

### Core Coordinator (`core.rs`)

- Coordinates AST node dispatching across specialized sub-engines.
- Manages recursive module loading, protecting against circular imports.
- Formulates quad-state truth projections.

---

## 4. Canonical Four-Block Layout

Every source file in the Evaluator subsystem strictly follows the **Canonical 4-Block Structure**:

1. **Block 1: Metadata Block**: File scope, scripture anchors, and AI/developer context rationale.
2. **Block 2: Opening Block**: Core imports, struct/enum definitions, and baseline initializers.
3. **Block 3: Body Block**: Business logic, operations, and trait implementations.
4. **Block 4: Closing Block**: Display formatting, default traits, and verification unit tests.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/connectives.rs

RS | 160 LINES | 599 WORDS | 7.1 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Logical Connectives, Adversative Caveats & Transactional Commit Guards )
// ================================================================================
//! OmniCode Runtime: Logical Connectives & Module Imports
//!
//! # Architectural Scope & Purpose:
//! Evaluates balanced ternary logical connectives:
//! - `! BUT`: Adversative caveat gate with automatic transactional rollback on failure.
//! - `& AND`: Conjunction ($A \land B$) under QuadState truth algebra.
//! - `| OR`: Disjunction ($A \lor B$) with short-circuit evaluation.
//! - `-- NOT`: Balanced ternary truth inversion.
//! - `:: IN_RELATION_TO`: Namespace resolution and module importation.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Caveat Gate Soundness**: The `! BUT` gate enforces strict reversible computing.
//!    If the caveat evaluates to `-1:ROLLBACK`, the entire speculative transaction is unwound.
// ================================================================================

use crate::ast::nodes::{ASTNode, ConnectiveNode};
use crate::core::operators::Connective;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::FailpointType;
use crate::runtime::environment::RuntimeValue;
use crate::runtime::evaluator::core::Evaluator;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Connective Evaluator Dispatcher Declaration )
// ================================================================================

/// Evaluates a `ConnectiveNode` under balanced ternary QuadState logic.
pub fn eval_connective(
    evaluator: &mut Evaluator,
    conn: &ConnectiveNode,
) -> Result<RuntimeValue, String> {
    match conn.connective {
        // ------------------------------------------------------------------------
        // Connective::But (! BUT) - Adversative Transactional Caveat
        // ------------------------------------------------------------------------
        Connective::But => {
            evaluator.env.begin_transaction();
            let action_res = evaluator.eval(&conn.left)?;
            let caveat_res = evaluator.eval(&conn.right)?;

            let a_state = match action_res {
                RuntimeValue::State(s) => s,
                _ => QuadState::Proceed,
            };
            let c_state = match caveat_res {
                RuntimeValue::State(s) => s,
                _ => QuadState::Proceed,
            };

            let final_state = a_state.but(c_state);

            if final_state == QuadState::Rollback {
                evaluator.env.rollback_transaction()?;
                evaluator.debugger.logger.failpoint(
                    FailpointType::CaveatVeto,
```

```
                    "EVALUATOR",
                    "Action vetoed by ! BUT caveat; state transaction rolled back",
                );
            } else if final_state == QuadState::Halt {
                evaluator.env.rollback_transaction()?;
                evaluator.debugger.logger.failpoint(
                    FailpointType::NullTrap,
                    "EVALUATOR",
                    "Action aborted by NULL caveat",
                );
            } else {
                evaluator.env.commit_transaction();
            }

            Ok(RuntimeValue::State(final_state))
        }

        // ------------------------------------------------------------------
        // Connective::And (& AND) - Conjunction
        // ------------------------------------------------------------------
        Connective::And => {
            let l = evaluator.eval_to_state(&conn.left)?;
            let r = evaluator.eval_to_state(&conn.right)?;
            Ok(RuntimeValue::State(l.and(r)))
        }

        // ------------------------------------------------------------------
        // Connective::Or (| OR) - Disjunction with Short-Circuit
        // ------------------------------------------------------------------
        Connective::Or => {
            let l = evaluator.eval_to_state(&conn.left)?;
            if l == QuadState::Proceed {
                return Ok(RuntimeValue::State(QuadState::Proceed));
            }
            let r = evaluator.eval_to_state(&conn.right)?;
            Ok(RuntimeValue::State(l.or(r)))
        }

        // ------------------------------------------------------------------
        // Connective::Not (-- NOT) - Ternary Truth Inversion
        // ------------------------------------------------------------------
        Connective::Not => {
            let r = evaluator.eval_to_state(&conn.right)?;
            Ok(RuntimeValue::State(r.invert()))
        }

        // ------------------------------------------------------------------
        // Connective::InRelationTo (:) - Module Import & Ontological Relation
        // ------------------------------------------------------------------
        Connective::InRelationTo => {
            if let ASTNode::Identifier(ref name) = *conn.left {
                if name == "import" {
                    let mut mod_names = Vec::new();
                    evaluator.extract_module_names(&conn.right, &mut mod_names);
                    for m in mod_names {
                        evaluator.import_module(&m)?;
                    }
                    return Ok(RuntimeValue::State(QuadState::Proceed));
                }
            }

            let right_res = evaluator.eval(&conn.right)?;
            if let Some(left_sym) =
                crate::runtime::evaluator::universal::extract_symbol_name(&conn.left)
            {
                if left_sym.starts_with("ROLL") {
                    evaluator.env.set(&left_sym, right_res.clone());
                    return Ok(right_res);
                }
            }
            evaluator.eval(&conn.left)
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
```

```
// ============================================================================
// Connectives orchestrate logical flow and transaction bounds.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Logical connectives layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/core.rs RS 728 LINES 2384 WORDS 32.9 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Evaluator Core Dispatcher, AST Execution Loop & Module Loader )
// ================================================================================
//! OmniCode Runtime: Evaluator Core Engine
//!
//! # Architectural Scope & Purpose:
//! Serves as the central execution orchestrator of the OmniCode VM. Coordinates
//! AST node dispatch across the specialized sub-engines:
//! - SVO Verbs & Memory Vessels (`verbs.rs`)
//! - Relational Algebra & Bounds (`relations.rs`)
//! - Directional Causal Propagation (`vectors.rs`)
//! - Logical Connectives & Adversative Caveats (`connectives.rs`)
//! - Universal Decoders & Trit Converters (`universal/decoder.rs`)
//!
//! Also manages dynamic module resolution and topological imports across `.omni`
//! reels and `.tape` ribbons.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Modular Architecture**: Moving sub-engines into dedicated modules eliminates
//!    monolithic complexity while guaranteeing complete unit test coverage.
// ================================================================================

use std::collections::HashSet;
use std::path::PathBuf;

use crate::ast::nodes::*;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::FailpointType;
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::{Environment, RuntimeValue};
use crate::runtime::evaluator::universal::extract_symbol_name;
use crate::runtime::evaluator::relations::eval_relation;
use crate::runtime::evaluator::vectors::eval_directional;
use crate::runtime::evaluator::connectives::eval_connective;
use crate::runtime::evaluator::verbs::eval_svo_verb_or_vessel;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Evaluator Struct Definition & Initialization )
// ================================================================================

/// Central execution engine coordinating AST evaluation, transactions, and modules.
pub struct Evaluator {
    pub env: Environment,
    pub debugger: OmniDebugger,
    pub loaded_modules: HashSet<String>,
}

impl Evaluator {
    /// Creates a new Evaluator with the given environment and debugger.
    pub fn new(env: Environment, debugger: OmniDebugger) -> Self {
        Self {
            env,
            debugger,
            loaded_modules: HashSet::new(),
        }
    }
}
```

```
// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Main Evaluation Loop, Module Loader & Helper Dispatchers )
// ============================================================================

impl Evaluator {
    /// Evaluates an AST node and yields a `RuntimeValue` or runtime error.
    pub fn eval(&mut self, node: &ASTNode) -> Result<RuntimeValue, String> {
        match node {
            // ------------------------------------------------------------------
            // Literals & Primitives
            // ------------------------------------------------------------------
            ASTNode::Ternary(state) => {
                self.debugger.inspect_state(*state, "TernaryLiteral");
                Ok(RuntimeValue::State(*state))
            }

            ASTNode::Number(val) => Ok(RuntimeValue::Number(*val)),

            ASTNode::StringLit(s) => Ok(RuntimeValue::Text(s.clone())),

            ASTNode::Identifier(name) => {
                if let Some(val) = self.env.get(name) {
                    Ok(val.clone())
                } else if name.eq_ignore_ascii_case("proceed") || name.eq_ignore_ascii_case("yes") {
                    Ok(RuntimeValue::State(QuadState::Proceed))
                } else if name.eq_ignore_ascii_case("rollback") || name.eq_ignore_ascii_case("no") {
                    Ok(RuntimeValue::State(QuadState::Rollback))
                } else if name.eq_ignore_ascii_case("await") {
                    Ok(RuntimeValue::State(QuadState::Await))
                } else if name.eq_ignore_ascii_case("halt") {
                    Ok(RuntimeValue::State(QuadState::Halt))
                } else {
                    self.debugger.logger.log("IDENTIFIER", format!("Symbolic ontological reference: '{name}'"));
                    Ok(RuntimeValue::Text(name.clone()))
                }
            }

            ASTNode::Polarity(p) => {
                let formatted = match p.position {
                    crate::core::operators::AffixPosition::Prefix => format!("{}{}", p.polarity.symbol(), p.name),
                    crate::core::operators::AffixPosition::Postfix => format!("{}{}", p.name, p.polarity.symbol()),
                };
                self.debugger.logger.log("POLARITY", format!("Evaluated polarity entity: {formatted}"));
                if p.name.eq_ignore_ascii_case("proceed") {
                    Ok(RuntimeValue::State(QuadState::Proceed))
                } else if p.name.eq_ignore_ascii_case("rollback") {
                    Ok(RuntimeValue::State(QuadState::Rollback))
                } else if p.name.eq_ignore_ascii_case("await") {
                    Ok(RuntimeValue::State(QuadState::Await))
                } else if p.name.eq_ignore_ascii_case("halt") {
                    Ok(RuntimeValue::State(QuadState::Halt))
                } else if let Some(val) = self.env.get(&formatted) {
                    Ok(val.clone())
                } else if let Some(val) = self.env.get(&p.name) {
                    Ok(val.clone())
                } else {
                    Ok(RuntimeValue::Text(formatted))
                }
            }

            ASTNode::BoundaryCast(bc) => {
                let (open, close) = bc.boundary.delimiters();
                let cast_str = format!("{}{}{}", open, bc.polarity.symbol(), close);
                self.debugger.logger.log("CAST", format!("Evaluated boundary cast: {cast_str}"));
                Ok(RuntimeValue::Text(cast_str))
            }

            ASTNode::Aside(aside) => {
                self.debugger.logger.log("ASIDE", format!("Evaluating Aside context: '{}'", aside.intent));
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ------------------------------------------------------------------
            // Boundary Scopes & Transaction Wrapping
            // ------------------------------------------------------------------
            ASTNode::Boundary(b) => {
```

```
                // Check if this boundary is a pure symbol reference (like [ ROLL 1 ], [ Verify ])
                if let Some(sym_name) = extract_symbol_name(node) {
                    if let Some(val) = self.env.get(&sym_name) {
                        return Ok(val.clone());
                    }
                }

                self.env.begin_transaction();
                let mut last_val = RuntimeValue::State(QuadState::Proceed);

                for el in &b.elements {
                    last_val = self.eval(el)?;
                    if let RuntimeValue::State(s) = last_val {
                        if s == QuadState::Rollback {
                            self.env.rollback_transaction()?;
                            self.debugger.logger.failpoint(
                                FailpointType::RollbackTriggered,
                                "EVALUATOR",
                                format!("Boundary {} unwound due to -1 (Rollback)", b.boundary_type),
                            );
                            return Ok(RuntimeValue::State(QuadState::Rollback));
                        } else if s == QuadState::Halt {
                            self.debugger.logger.failpoint(
                                FailpointType::NullTrap,
                                "EVALUATOR",
                                format!("Boundary {} trapped by NULL / Void", b.boundary_type),
                            );
                            return Ok(RuntimeValue::State(QuadState::Halt));
                        }
                    }
                }

                self.env.commit_transaction();
                Ok(last_val)
            }

            // ---------------------------------------------------------------
            // Entity Definitions & SVO Action Verbs
            // ---------------------------------------------------------------
            ASTNode::Entity(entity) => {
                // Try dispatching as an SVO Action Verb or Memory Vessel
                if let Some(verb_res) = eval_svo_verb_or_vessel(self, entity)? {
                    return Ok(verb_res);
                }

                let is_imm = entity.definition.boundary_type.is_immutable();
                if is_imm {
                    self.debugger.register_immutable(&entity.name);
                } else {
                    self.debugger.assert_mutation_permitted(&entity.name)?;
                }

                if let Some(ref aside) = entity.aside {
                    self.debugger.logger.log(
                        "ASIDE",
                        format!("Entity '{}' attached intent: ({})", entity.name, aside.intent),
                    );
                }

                let val = self.eval(&ASTNode::Boundary(entity.definition.clone()))?;
                self.env.set(&entity.name, val.clone());
                self.debugger.logger.log("ENTITY", format!("Defined entity '{}' = {:?}", entity.name, val));
                Ok(val)
            }

            // ---------------------------------------------------------------
            // Relational Algebra
            // ---------------------------------------------------------------
            ASTNode::Relation(rel) => eval_relation(self, rel),

            // ---------------------------------------------------------------
            // Logical Connectives & Adversative Caveats
            // ---------------------------------------------------------------
            ASTNode::Connective(conn) => eval_connective(self, conn),

            // ---------------------------------------------------------------
            // Directional Causal Propagation
            // ---------------------------------------------------------------
```

```
            ASTNode::Directional(dir) => eval_directional(self, dir),

            // ---------------------------------------------------------------
            // Scalar Arithmetic
            // ---------------------------------------------------------------
            ASTNode::Arithmetic(arith) => {
                let left_val = self.eval(&arith.left)?;
                let right_val = self.eval(&arith.right)?;

                match (&left_val, &right_val) {
                    (RuntimeValue::Number(l), RuntimeValue::Number(r)) => {
                        let res = match arith.op {
                            ArithmeticOp::Add => l + r,
                            ArithmeticOp::Sub => l - r,
                            ArithmeticOp::Mul => l * r,
                            ArithmeticOp::Div => {
                                if *r == 0.0 {
                                    self.debugger.logger.failpoint(
                                        FailpointType::NullTrap,
                                        "EVALUATOR",
                                        "Division by zero traps as NULL/Void",
                                    );
                                    return Ok(RuntimeValue::State(QuadState::Halt));
                                }
                                l / r
                            }
                        };
                        Ok(RuntimeValue::Number(res))
                    }
                    _ => {
                        self.debugger.logger.log(
                            "ARITHMETIC",
                            format!("Symbolic arithmetic between {:?} and {:?}", left_val, right_val),
                        );
                        Ok(right_val)
                    }
                }
            }

            // ---------------------------------------------------------------
            // Unary Inversion
            // ---------------------------------------------------------------
            ASTNode::UnaryNot(un) => {
                let s = self.eval_to_state(&un.operand)?;
                Ok(RuntimeValue::State(s.invert()))
            }

            // ---------------------------------------------------------------
            // Program Block Execution
            // ---------------------------------------------------------------
            ASTNode::Program(prog) => {
                let mut last = RuntimeValue::State(QuadState::Proceed);
                for stmt in &prog.statements {
                    last = self.eval(stmt)?;
                }
                Ok(last)
            }

            // ---------------------------------------------------------------
            // Module Declaration
            // ---------------------------------------------------------------
            ASTNode::Module(module) => {
                self.debugger.logger.log("MODULE", format!("Mounting Module: '{}'", module.name));
                self.env.set("Module", RuntimeValue::Text(module.name.clone()));
                if let Some(ref aside) = module.aside {
                    self.debugger.logger.log("ASIDE", format!("Module intent: ({})", aside.intent));
                }
                for field in &module.fields {
                    self.eval(field)?;
                }
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ---------------------------------------------------------------
            // Process Tape Declaration
            // ---------------------------------------------------------------
            ASTNode::Tape(tape) => {
                self.debugger.logger.log("TAPE", format!("Mounting Process Tape: '{}'", tape.name));
```

```
                self.env.set("Tape", RuntimeValue::Text(tape.name.clone()));
                if let Some(ref aside) = tape.aside {
                    self.debugger.logger.log("ASIDE", format!("Tape intent: ({})", aside.intent));
                }
                for clause in &tape.clauses {
                    self.eval(clause)?;
                }
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ---------------------------------------------------------------
            // 4-Block Section Execution
            // ---------------------------------------------------------------
            ASTNode::Section(section) => {
                self.debugger.logger.log("SECTION", format!("Entering 4-Block Section: {}", section.kind.name()));
                if let Some(ref aside) = section.aside {
                    self.debugger.logger.log("ASIDE", format!("Section intent: ({})", aside.intent));
                }
                let mut last = RuntimeValue::State(QuadState::Proceed);
                for element in &section.elements {
                    last = self.eval(element)?;
                }
                Ok(last)
            }

            // ---------------------------------------------------------------
            // Discrete Operational Roll Execution
            // ---------------------------------------------------------------
            ASTNode::Roll(roll) => {
                self.debugger.logger.log("ROLL", format!("Executing ROLL {}: {}", roll.id, roll.label));
                if let Some(ref aside) = roll.aside {
                    self.debugger.logger.log("ASIDE", format!("Roll intent: ({})", aside.intent));
                }
                let val = self.eval(&roll.action)?;
                self.env.set(&format!("ROLL {}", roll.id), val.clone());
                Ok(val)
            }

            // ---------------------------------------------------------------
            // Closing Fourfold Commit Status Gate
            // ---------------------------------------------------------------
            ASTNode::CommitStatus(cs) => {
                self.debugger.logger.log("COMMIT", "Evaluating Fourfold Commit Status Gate");
                if let Some(ref aside) = cs.aside {
                    self.debugger.logger.log("ASIDE", format!("Commit intent: ({})", aside.intent));
                }
                let mut status = QuadState::Proceed;
                for case in &cs.cases {
                    let val = self.eval(case)?;
                    let state = val.to_quad_state();
                    if state == QuadState::Proceed {
                        status = QuadState::Proceed;
                        break;
                    } else if state == QuadState::Rollback {
                        status = QuadState::Rollback;
                    }
                }
                Ok(RuntimeValue::State(status))
            }

            // ---------------------------------------------------------------
            // Cardinal Interrogatives (WHO, WHAT, WHEN, WHERE, WHY, HOW)
            // ---------------------------------------------------------------
            ASTNode::Cardinal(cardinal) => {
                let kw = cardinal.kind.keyword();
                let val = self.eval(&cardinal.payload)?;
                self.debugger.logger.log("CARDINAL", format!("{kw} clause registered: {:?}", val));
                self.env.set(kw, val.clone());
                Ok(val)
            }

            // ---------------------------------------------------------------
            // Operational SVO Verbs (INSCRIBE, TRANSDUCE, GUARD, PROJECT, SEQUENCE)
            // ---------------------------------------------------------------
            ASTNode::Verb(v) => {
                self.debugger.logger.log("VERB", format!("Executing SVO Action Verb: {}", v.verb.name()));
                let obj_val = self.eval(&v.object)?;
                if let Some(ref target) = v.target {
```

```
                    if let Some(target_name) = extract_symbol_name(target) {
                        self.env.set(&target_name, obj_val.clone());
                    }
                }
                Ok(obj_val)
            }

            // ------------------------------------------------------------------
            // Adversative Guard Assertions (! GUARD)
            // ------------------------------------------------------------------
            ASTNode::Guard(g) => {
                let cond_val = self.eval(&g.condition)?;
                let cond_state = cond_val.to_quad_state();
                if cond_state == QuadState::Proceed {
                    self.eval(&g.on_proceed)
                } else if let Some(ref fallback) = g.on_fallback {
                    self.eval(fallback)
                } else {
                    Ok(RuntimeValue::State(QuadState::Rollback))
                }
            }

            // ------------------------------------------------------------------
            // MMIO Hardware Bus Locators (::@)
            // ------------------------------------------------------------------
            ASTNode::Hardware(hw) => {
                let target_val = self.eval(&hw.target)?;
                self.debugger.logger.log("HARDWARE", format!("MMIO Address 0x{:X} mapped to {:?}", hw.address,
target_val));
                Ok(RuntimeValue::Number(hw.address as f64))
            }

            // ------------------------------------------------------------------
            // Invariant Ontological Type Definitions
            // ------------------------------------------------------------------
            ASTNode::InvariantType(inv) => {
                self.debugger.logger.log("INVARIANT", format!("Registered Invariant Type: <{}>", inv.name));
                self.env.set(&inv.name, RuntimeValue::Text(format!("<{}>", inv.name)));
                if let Some(ref aside) = inv.aside {
                    self.debugger.logger.log("ASIDE", format!("Type intent: ({})", aside.intent));
                }
                for invariant in &inv.invariants {
                    self.eval(invariant)?;
                }
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ------------------------------------------------------------------
            // Ontological Analogy Proportions (A : B :: C : D)
            // ------------------------------------------------------------------
            ASTNode::Analogy(_) => {
                self.debugger.logger.log("ANALOGY", "Evaluating ontological proportion");
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ------------------------------------------------------------------
            // Dual Substrate Equivalence (Bits <=> Trits)
            // ------------------------------------------------------------------
            ASTNode::Substrate(sub) => {
                self.debugger.logger.log(
                    "SUBSTRATE",
                    format!("Dual substrate: {} physical bits <=> {} virtual trits", sub.physical_bits,
sub.virtual_trits),
                );
                Ok(RuntimeValue::State(QuadState::Proceed))
            }

            // ------------------------------------------------------------------
            // Canonical Exemplar Mappings (Scalar <=> Trits)
            // ------------------------------------------------------------------
            ASTNode::Exemplar(ex) => {
                self.debugger.logger.log(
                    "EXEMPLAR",
                    format!("Canonical exemplar: {} <=> {:?}", ex.scalar, ex.trits),
                );
                Ok(RuntimeValue::State(QuadState::Proceed))
            }
        }
```

```
    }

    /// Helper evaluating a node and coercing its result to a `QuadState`.
    pub fn eval_to_state(&mut self, node: &ASTNode) -> Result<QuadState, String> {
        let val = self.eval(node)?;
        Ok(val.to_quad_state())
    }

    /// Recursively extracts module identifier names from an AST node.
    pub fn extract_module_names(&self, node: &ASTNode, names: &mut Vec<String>) {
        match node {
            ASTNode::Identifier(name) => {
                if !name.is_empty() {
                    names.push(name.clone());
                }
            }
            ASTNode::Boundary(b) => {
                for el in &b.elements {
                    self.extract_module_names(el, names);
                }
            }
            ASTNode::Connective(conn) => {
                self.extract_module_names(&conn.left, names);
                self.extract_module_names(&conn.right, names);
            }
            _ => {}
        }
    }

    /// Resolves an omnicode module name into a physical file path.
    pub fn resolve_module_path(&self, module_name: &str) -> Option<PathBuf> {
        let parts: Vec<&str> = module_name.split('.').collect();
        let rel_slash = parts.join("/").to_lowercase();
        let rel_dot = module_name.to_lowercase();

        let candidates = [
            format!("lib/{rel_slash}.omni"),
            format!("lib/{rel_dot}.omni"),
            format!("lexicon/{rel_slash}.omni"),
            format!("lexicon/{rel_dot}.omni"),
            format!("{rel_slash}.omni"),
            format!("{rel_dot}.omni"),
            format!("examples/{rel_slash}.omni"),
            format!("examples/{rel_dot}.omni"),
        ];

        for cand in &candidates {
            let path = crate::core::filetype::resolve_track_path(cand);
            if path.exists() {
                return Some(path);
            }
        }
        None
    }

    /// Imports and executes an omnicode module into the active environment.
    pub fn import_module(&mut self, module_name: &str) -> Result<RuntimeValue, String> {
        if self.loaded_modules.contains(module_name) {
            self.debugger.logger.log(
                "IMPORT",
                format!("Module '{module_name}' already loaded; skipping redundant load"),
            );
            return Ok(RuntimeValue::State(QuadState::Proceed));
        }

        let path = match self.resolve_module_path(module_name) {
            Some(p) => p,
            None => {
                let msg = format!("Module '{module_name}' could not be resolved in lib/ or working directory");
                self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "IMPORT", &msg);
                return Err(format!("FAILPOINT: {msg}"));
            }
        };

        let content = std::fs::read_to_string(&path).map_err(|e| {
            let msg = format!("Failed to read module '{module_name}' at {:?}: {e}", path);
            self.debugger.logger.failpoint(FailpointType::SyntaxAnomaly, "IMPORT", &msg);
            format!("FAILPOINT: {msg}")
```

```
        })?;

        self.debugger.logger.log(
            "IMPORT",
            format!("Loading module '{module_name}' from {:?}", path),
        );

        // Track module load immediately to protect against circular imports
        self.loaded_modules.insert(module_name.to_string());

        let mut lexer = Lexer::new(&content, self.debugger.logger.clone());
        let tokens = lexer.tokenize()?;

        let mut parser = Parser::new(tokens, self.debugger.clone());
        let prog = parser.parse_program()?;

        let result = self.eval(&ASTNode::Program(prog))?;

        self.debugger.logger.log(
            "IMPORT",
            format!("Successfully loaded module '{module_name}' (=> {:?})", result),
        );

        Ok(RuntimeValue::State(QuadState::Proceed))
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Execution Invariant Proofs, Scripture Witness & In-Module Verification Tests )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::logger::DiagnosticLogger;

    fn create_test_evaluator() -> Evaluator {
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 100);
        let env = Environment::new();
        Evaluator::new(env, debugger)
    }

    /// Witness Proof 1: Exodus 31:18 KJV
    /// Bijective Radix-3 Transduction (<=>) Canonical Exemplar Letter 'A' (65).
    #[test]
    fn test_bijective_transduction_exemplar() {
        let mut eval = create_test_evaluator();
        let src = "65 <=> [ -1, +1, +1, -1, +1 ]";
        let mut lexer = Lexer::new(src, eval.debugger.logger.clone());
        let tokens = lexer.tokenize().expect("Lexing must succeed");
        let mut parser = Parser::new(tokens, eval.debugger.clone());
        let prog = parser.parse_program().expect("Parsing must succeed");

        let result = eval.eval(&ASTNode::Program(prog)).expect("Eval must succeed");
        assert_eq!(
            result,
            RuntimeValue::State(QuadState::Proceed),
            "Exemplar 'A' (65 <=> [ -1, +1, +1, -1, +1 ]) must evaluate to +1:PROCEED"
        );
    }

    /// Witness Proof 2: Proverbs 24:27 KJV
    /// Prepares and proves the SVO action verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE)
    /// in isolation before constructing higher-level systems.
    #[test]
    fn test_svo_verbs_character_lifecycle() {
        let mut eval = create_test_evaluator();

        // 1. Inscribe incoming glyph 'A' (65)
        eval.env.set("Glyph", RuntimeValue::Text("A".into()));
        let node_inscribe = ASTNode::Entity(EntityNode {
            name: "INSCRIBE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Cell".into())],
                aside: None,
```

```
                line: 1,
                col: 1,
            },
            aside: None,
            context: None,
            line: 1,
            col: 1,
        });
        let res_inscribe = eval.eval(&node_inscribe).expect("INSCRIBE must succeed");
        assert!(matches!(res_inscribe, RuntimeValue::Character { glyph: 65, .. }));

        // 2. Transduce Character into Cell trits
        let node_transduce = ASTNode::Entity(EntityNode {
            name: "TRANSDUCE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Cell`".into())],
                aside: None,
                line: 2,
                col: 1,
            },
            aside: None,
            context: None,
            line: 2,
            col: 1,
        });
        let res_transduce = eval.eval(&node_transduce).expect("TRANSDUCE must succeed");
        assert!(matches!(res_transduce, RuntimeValue::Cell(_)));

        // 3. Project Character to Target (simulating terminal stdout / INTO port)
        let node_project = ASTNode::Entity(EntityNode {
            name: "PROJECT".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Target".into())],
                aside: None,
                line: 3,
                col: 1,
            },
            aside: None,
            context: None,
            line: 3,
            col: 1,
        });
        let res_project = eval.eval(&node_project).expect("PROJECT must succeed");
        assert!(matches!(res_project, RuntimeValue::Character { glyph: 65, .. }));
        assert_eq!(eval.env.read_into(), RuntimeValue::from_glyph(65));

        // 4. Sequence Character into continuous Text ribbon
        let node_sequence = ASTNode::Entity(EntityNode {
            name: "SEQUENCE".into(),
            definition: BoundaryNode {
                boundary_type: crate::core::boundaries::BoundaryType::Denotation,
                elements: vec![ASTNode::Identifier("Text".into())],
                aside: None,
                line: 4,
                col: 1,
            },
            aside: None,
            context: None,
            line: 4,
            col: 1,
        });
        let res_seq = eval.eval(&node_sequence).expect("SEQUENCE must succeed");
        assert_eq!(res_seq, RuntimeValue::Text("A".into()));
    }

    /// Witness Proof 3: Proverbs 22:28 KJV
    /// Preserves ancient landmarks by proving that a character input from the terminal
    /// (%from / 0x03F8) flows end-to-end through the Character tape, echoes outward
    /// to the terminal screen (%into / 0x000B8000), and commits in +1:PROCEED.
    #[test]
    fn test_terminal_character_echo_end_to_end() {
        let mut eval = create_test_evaluator();

        // Simulate incoming terminal keystroke 'K' (ASCII 75) arriving on %from
        eval.env.write_from(RuntimeValue::from_glyph(b'K'));
        eval.env.set("Glyph", RuntimeValue::Text("K".into()));
```

```
        // Execute Inscribe
        let inscribe_src = "{ Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]";
        let tokens1 = Lexer::new(inscribe_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog1 = Parser::new(tokens1, eval.debugger.clone()).parse_program().unwrap();
        let res1 = eval.eval(&ASTNode::Program(prog1)).expect("Roll 1 inscribe must succeed");
        assert_eq!(res1, RuntimeValue::State(QuadState::Proceed));

        // Execute Transduce
        let transduce_src = "[ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]";
        let tokens2 = Lexer::new(transduce_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog2 = Parser::new(tokens2, eval.debugger.clone()).parse_program().unwrap();
        let res2 = eval.eval(&ASTNode::Program(prog2)).expect("Roll 2 transduce must succeed");
        assert_eq!(res2, RuntimeValue::State(QuadState::Proceed));

        // Execute Project (Hardware Echo to %into / Target)
        let project_src = "[ Character` } --> PROJECT [ Target ] = [ Target` }";
        let tokens3 = Lexer::new(project_src, eval.debugger.logger.clone()).tokenize().unwrap();
        let prog3 = Parser::new(tokens3, eval.debugger.clone()).parse_program().unwrap();
        let res3 = eval.eval(&ASTNode::Program(prog3)).expect("Roll 4 project must succeed");
        assert_eq!(res3, RuntimeValue::State(QuadState::Proceed));

        // Assert that the echoed character arrived in the %into register
        let into_val = eval.env.read_into();
        if let RuntimeValue::Character { glyph, .. } = into_val {
            assert_eq!(glyph, b'K', "Echoed character glyph must exactly match terminal input 'K'");
        } else {
            panic!("Expected RuntimeValue::Character in %into, got {:?}", into_val);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/mod.rs RS 60 LINES 375 WORDS 3.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Evaluator Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Runtime: Evaluator Subsystem Facade
//!
//! # Architectural Scope & Purpose:
//! Modularizes the AST execution engine into specialized domains:
//! - `universal`: Pure AST pattern extractors, trit sequence decoders, and Horner algorithms.
//! - `relations`: Relational algebra (`=`, `!=`, `~=`, `<=>`, `<=`, `>=`) and bounds.
//! - `vectors`: Directional causal propagation vectors (`-->`, `<--`, `^--`, `v--`).
//! - `connectives`: Quad-State logical gates (`! BUT`, `& AND`, `| OR`, `-- NOT`, `::`).
//! - `verbs`: SVO action verbs (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`) and vessels.
//! - `core`: The central `Evaluator` coordinator and module loader.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Complete Decoupling**: Modularizing the evaluator prevents monolith regressions
//!    while providing individual sub-engines for specialized testing.
//! 2. **Backward Compatibility**: Re-exports `Evaluator` at the module root, ensuring zero
//!    disruptions to existing test suites, REPL, and CLI commands.
// ============================================================================

pub mod universal;
pub mod relations;
pub mod vectors;
pub mod connectives;
pub mod verbs;
pub mod core;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Public Evaluator Access )
// ============================================================================
pub use core::Evaluator;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// The Evaluator executes on top of the Transactional Environment substrate.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Evaluator subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/relations.rs

RS | 313 LINES | 1183 WORDS | 15.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Relational Algebra, Being Relations & Bijective Equivalence Proofs )
// ================================================================================
//! OmniCode Runtime: Relational Algebra & Being Relations
//!
//! # Architectural Scope & Purpose:
//! Evaluates Being Relations across dual-substrate operands:
//! - `Is` (`=`): Observational glyph and value equality, and state target assignment.
//! - `IsNot` (`!=`): Inequality and difference verification.
//! - `ToBeLike` (`~=`): Ontological type compatibility.
//! - `Equivalence` (`<=>`): Bijective Radix-3 numeric-to-trit transduction.
//! - `LessThan` (`<`), `GreaterThan` (`>`), `LessEqual` (`<=`), `GreaterEqual` (`>=`):
//!   Scalar numeric and universal physical memory capacity bounds (`capacity_in_cells`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Separation of Observation vs. Mutation**: Guard conditions like `{ Character' = Glyph }`
//!    perform relational equality tests using `is_glyph_equal` without mutating base state.
//! 2. **Capacity Bounds**: Expressions like `{ Character' <= [ 1 Cell ] }` leverage
//!    the universal `capacity_in_cells()` metric to guarantee memory integrity.
// ================================================================================

use crate::ast::nodes::{ASTNode, RelationNode};
use crate::core::cell::Cell;
use crate::core::operators::BeingRelation;
use crate::core::ternary::QuadState;
use crate::runtime::environment::RuntimeValue;
use crate::runtime::evaluator::core::Evaluator;
use crate::runtime::evaluator::universal::{
    extract_symbol_name,
    extract_trits_from_node,
    trits_to_integer,
};

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Relational Evaluation Dispatcher Declaration )
// ================================================================================

/// Evaluates a `RelationNode` under balanced ternary QuadState semantics.
pub fn eval_relation(evaluator: &mut Evaluator, rel: &RelationNode) -> Result<RuntimeValue, String> {
    let left_val = evaluator.eval(&rel.left)?;
    let right_val = evaluator.eval(&rel.right)?;

    let result = match rel.relation {
        // ------------------------------------------------------------------------
        // BeingRelation::Is (=)
        // ------------------------------------------------------------------------
        BeingRelation::Is => {
            // 1. Right-side assignment target (e.g. INSCRIBE [ Cell ] = [ Character` ])
            if let Some(target_ident) = extract_symbol_name(&rel.right) {
                let is_state_prime = target_ident.ends_with(['`', '\'']);
                let is_keyword = target_ident.eq_ignore_ascii_case("proceed")
                    || target_ident.eq_ignore_ascii_case("rollback")
                    || target_ident.eq_ignore_ascii_case("await")
                    || target_ident.eq_ignore_ascii_case("halt")
                    || target_ident.eq_ignore_ascii_case("yes")
                    || target_ident.eq_ignore_ascii_case("no")
                    || target_ident.eq_ignore_ascii_case("armed")
                    || target_ident.eq_ignore_ascii_case("tripped")
```

```
                    || target_ident.eq_ignore_ascii_case("even")
                    || target_ident.eq_ignore_ascii_case("odd");

                if is_state_prime || (!is_keyword && evaluator.env.get(&target_ident).is_none()) {
                    evaluator.env.set(&target_ident, left_val.clone());
                    return Ok(RuntimeValue::State(QuadState::Proceed));
                }
            }

            // 2. Left-side assignment/binding target (e.g. [ Bit_7 = "0" ], [ Status` = +1:PROCEED ])
            if let Some(target_ident) = extract_symbol_name(&rel.left) {
                let is_state_prime = target_ident.ends_with(['`', '\'']);
                let is_keyword = target_ident.eq_ignore_ascii_case("proceed")
                    || target_ident.eq_ignore_ascii_case("rollback")
                    || target_ident.eq_ignore_ascii_case("await")
                    || target_ident.eq_ignore_ascii_case("halt")
                    || target_ident.eq_ignore_ascii_case("yes")
                    || target_ident.eq_ignore_ascii_case("no")
                    || target_ident.eq_ignore_ascii_case("armed")
                    || target_ident.eq_ignore_ascii_case("tripped")
                    || target_ident.eq_ignore_ascii_case("even")
                    || target_ident.eq_ignore_ascii_case("odd");

                if (is_state_prime && matches!(right_val, RuntimeValue::State(_)))
                    || (!is_keyword && evaluator.env.get(&target_ident).is_none())
                {
                    evaluator.env.set(&target_ident, right_val.clone());
                    return Ok(RuntimeValue::State(QuadState::Proceed));
                }
            }

            // 3. Relational comparison using universal equality & glyph checks
            if left_val.is_glyph_equal(&right_val) {
                QuadState::Proceed
            } else if let (RuntimeValue::Number(n), RuntimeValue::Text(t)) = (&left_val, &right_val) {
                let is_even = (n.round() as i64) % 2 == 0;
                if (t.eq_ignore_ascii_case("even") && is_even) || (t.eq_ignore_ascii_case("odd") && !is_even) {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else if left_val == right_val {
                QuadState::Proceed
            } else {
                QuadState::Rollback
            }
        }

        // ---------------------------------------------------------------------
        // BeingRelation::IsNot (!=) & BeingRelation::ToBeNot (=!)
        // ---------------------------------------------------------------------
        BeingRelation::IsNot | BeingRelation::ToBeNot => {
            if left_val.is_glyph_equal(&right_val) {
                QuadState::Rollback
            } else if let (RuntimeValue::Number(n), RuntimeValue::Text(t)) = (&left_val, &right_val) {
                let is_even = (n.round() as i64) % 2 == 0;
                if (t.eq_ignore_ascii_case("even") && !is_even) || (t.eq_ignore_ascii_case("odd") && is_even) {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else if left_val != right_val {
                QuadState::Proceed
            } else {
                QuadState::Rollback
            }
        }

        // ---------------------------------------------------------------------
        // BeingRelation::ToBeLike (~=)
        // ---------------------------------------------------------------------
        BeingRelation::ToBeLike => match (&left_val, &right_val) {
            (RuntimeValue::Number(_), RuntimeValue::Number(_)) => QuadState::Proceed,
            (RuntimeValue::Text(_), RuntimeValue::Text(_)) => QuadState::Proceed,
            (RuntimeValue::State(_), RuntimeValue::State(_)) => QuadState::Proceed,
            (RuntimeValue::Character { .. }, RuntimeValue::Character { .. }) => QuadState::Proceed,
            _ => QuadState::Rollback,
        },
```

```
        // ---------------------------------------------------------------------
        // BeingRelation::Equivalence (<=>)
        // ---------------------------------------------------------------------
        BeingRelation::Equivalence => {
            // Bijective Radix-3 Transduction: Number <=> [ Trits ] (e.g. 65 <=> [ -1, +1, +1, -1, +1 ])
            if let (ASTNode::Number(num), Some(trits)) = (&*rel.left, extract_trits_from_node(&rel.right)) {
                let expected = *num as i64;
                let actual = trits_to_integer(&trits);
                if expected == actual {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else if let (Some(trits), ASTNode::Number(num)) = (extract_trits_from_node(&rel.left), &*rel.right) {
                let expected = *num as i64;
                let actual = trits_to_integer(&trits);
                if expected == actual {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else if let (RuntimeValue::Character { glyph, cell }, RuntimeValue::Cell(c)) = (&left_val, &right_val) {
                if cell == c || Cell::from_natural(*glyph).map_or(false, |calc| calc == *c) {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else if let (RuntimeValue::Cell(c), RuntimeValue::Character { glyph, cell }) = (&left_val, &right_val) {
                if cell == c || Cell::from_natural(*glyph).map_or(false, |calc| calc == *c) {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            } else {
                let l_sym = extract_symbol_name(&rel.left).unwrap_or_default();
                let r_sym = extract_symbol_name(&rel.right).unwrap_or_default();
                let l_txt = left_val.as_text().unwrap_or("");
                let r_txt = right_val.as_text().unwrap_or("");
                let has_bit = l_sym.contains("Bit")
                    || r_sym.contains("Bit")
                    || l_txt.contains("Bit")
                    || r_txt.contains("Bit");
                let has_trit = l_sym.contains("Trit")
                    || r_sym.contains("Trit")
                    || l_txt.contains("Trit")
                    || r_txt.contains("Trit");

                if has_bit && has_trit {
                    QuadState::Proceed
                } else if let (RuntimeValue::Text(t1), RuntimeValue::Text(t2)) = (&left_val, &right_val) {
                    if t1 == t2 {
                        QuadState::Proceed
                    } else {
                        QuadState::Rollback
                    }
                } else if left_val.is_glyph_equal(&right_val) {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
        }

        // ---------------------------------------------------------------------
        // BeingRelation::LessThan (<)
        // ---------------------------------------------------------------------
        BeingRelation::LessThan => match (&left_val, &right_val) {
            (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                if a < b { QuadState::Proceed } else { QuadState::Rollback }
            }
            _ => {
                if left_val.capacity_in_cells() < right_val.capacity_in_cells() {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
```

```
        },

        // ------------------------------------------------------------------
        // BeingRelation::GreaterThan (>)
        // ------------------------------------------------------------------
        BeingRelation::GreaterThan => match (&left_val, &right_val) {
            (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                if a > b { QuadState::Proceed } else { QuadState::Rollback }
            }
            _ => {
                if left_val.capacity_in_cells() > right_val.capacity_in_cells() {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
        },

        // ------------------------------------------------------------------
        // BeingRelation::LessEqual (<=)
        // ------------------------------------------------------------------
        BeingRelation::LessEqual => match (&left_val, &right_val) {
            (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                if a <= b { QuadState::Proceed } else { QuadState::Rollback }
            }
            (RuntimeValue::Character { .. }, RuntimeValue::Number(b)) => {
                if *b >= 1.0 { QuadState::Proceed } else { QuadState::Rollback }
            }
            (RuntimeValue::Character { .. }, RuntimeValue::Cell(_)) => {
                QuadState::Proceed
            }
            (RuntimeValue::Character { .. }, RuntimeValue::Text(t)) => {
                if t.contains("Cell") || t.contains("Byte") || t.contains('1') {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
            _ => {
                if left_val.capacity_in_cells() <= right_val.capacity_in_cells() {
                    QuadState::Proceed
                } else if left_val == right_val {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
        },

        // ------------------------------------------------------------------
        // BeingRelation::GreaterEqual (>=)
        // ------------------------------------------------------------------
        BeingRelation::GreaterEqual => match (&left_val, &right_val) {
            (RuntimeValue::Number(a), RuntimeValue::Number(b)) => {
                if a >= b { QuadState::Proceed } else { QuadState::Rollback }
            }
            _ => {
                if left_val.capacity_in_cells() >= right_val.capacity_in_cells() {
                    QuadState::Proceed
                } else if left_val == right_val {
                    QuadState::Proceed
                } else {
                    QuadState::Rollback
                }
            }
        },
    };

    evaluator.debugger.inspect_state(result, "BeingRelation");
    Ok(RuntimeValue::State(result))
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// Relational evaluations return balanced ternary QuadState truth values.

```

```
// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ================================================================================
// Relational algebra layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/vectors.rs RS 254 LINES 929 WORDS 12.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Directional Causal Vectors, Flow Propagation & Silicon Register Routing )
// ============================================================================
//! OmniCode Runtime: Directional Causal Vectors
//!
//! # Architectural Scope & Purpose:
//! Evaluates directional causal vector flows:
//! - `ActUpon` (`-->`): Primary causal assertion, state advancement, and output port routing.
//! - `ActAgain` (`<--`): Retroaction, feedback loops, and memory vessel reads.
//! - `BuildsTo` (`^--`): Vertical hierarchical assembly.
//! - `BreaksDownFrom` (`v--`): Storehouse partitioning and memory allocation.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Causal Non-Corruption**: Causal action `A --> B` propels `A` into `B` without
//!    retroactively corrupting `A` with `B`'s status result.
//! 2. **Hardware Register Synchronization**: Writes targeting `%into` route directly
//!    to the outbound hardware port and buffer stream.
// ============================================================================

use crate::ast::nodes::{ASTNode, DirectionalNode};
use crate::core::cell::Cell;
use crate::core::operators::DirectionalVector;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::FailpointType;
use crate::runtime::assembler::MachineRegister;
use crate::runtime::environment::RuntimeValue;
use crate::runtime::evaluator::core::Evaluator;
use crate::runtime::evaluator::universal::extract_symbol_name;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Directional Vector Dispatcher Declaration )
// ============================================================================

/// Evaluates a `DirectionalNode` carrying causal propagation.
pub fn eval_directional(
    evaluator: &mut Evaluator,
    dir: &DirectionalNode,
) -> Result<RuntimeValue, String> {
    match dir.vector {
        // ---------------------------------------------------------------------
        // DirectionalVector::ActUpon (-->)
        // ---------------------------------------------------------------------
        DirectionalVector::ActUpon => {
            // 1. Indexed Vessel write: Value --> Vessel[Offset]
            if let ASTNode::Entity(ref target_entity) = *dir.target {
                if let Some(RuntimeValue::Vessel(mut vessel)) = evaluator.env.get(&target_entity.name).cloned() {
                    let src_val = evaluator.eval(&dir.source)?;
                    let offset_val = if let Some(first_el) = target_entity.definition.elements.first() {
                        evaluator.eval(first_el)?
                    } else {
                        RuntimeValue::Number(0.0)
                    };
                    let offset = match offset_val {
                        RuntimeValue::Number(n) => n as usize,
                        _ => 0,
                    };
                    let cell = match src_val {
                        RuntimeValue::Number(n) => Cell::from_natural(n as u8).unwrap_or_else(|_|
Cell::from_state(QuadState::Rollback)),
```

```
                        RuntimeValue::State(s) => Cell::from_state(s),
                        RuntimeValue::Cell(c) => c,
                        RuntimeValue::Character { cell, .. } => cell,
                        _ => Cell::from_state(QuadState::Proceed),
                    };
                    let write_res = vessel.write(offset, cell).unwrap_or(QuadState::Rollback);
                    evaluator.env.set(&target_entity.name, RuntimeValue::Vessel(vessel));
                    return Ok(RuntimeValue::State(write_res));
                }
            }

            // 2. Evaluate source entity
            let src_val = evaluator.eval(&dir.source)?;
            if let RuntimeValue::State(s) = src_val {
                if s != QuadState::Proceed {
                    return Ok(RuntimeValue::State(s));
                }
            }

            // If source is a glyph carrier (e.g. { Glyph ]), ensure Glyph is set in env
            if let Some(src_name) = extract_symbol_name(&dir.source) {
                if src_name == "Glyph" {
                    evaluator.env.set("Glyph", src_val.clone());
                }
            }

            // 3. Target Register or Named Variable Routing
            let target_symbol_opt = extract_symbol_name(&dir.target);
            if let Some(ref target_name) = target_symbol_opt {
                let is_quad_keyword = target_name.eq_ignore_ascii_case("proceed")
                    || target_name.eq_ignore_ascii_case("await")
                    || target_name.eq_ignore_ascii_case("rollback")
                    || target_name.eq_ignore_ascii_case("halt")
                    || target_name.eq_ignore_ascii_case("yes")
                    || target_name.eq_ignore_ascii_case("no");

                if target_name.eq_ignore_ascii_case("armed") {
                    evaluator.env.set("Tripwire", RuntimeValue::Text("ARMED".to_string()));
                    evaluator.env.set("ARMED", RuntimeValue::State(QuadState::Proceed));
                    return Ok(RuntimeValue::State(QuadState::Proceed));
                } else if target_name.eq_ignore_ascii_case("tripped") {
                    evaluator.env.set("Tripwire", RuntimeValue::Text("TRIPPED".to_string()));
                    evaluator.env.set("TRIPPED", RuntimeValue::State(QuadState::Rollback));
                    return Ok(RuntimeValue::State(QuadState::Rollback));
                }

                if !is_quad_keyword {
                    if let Some(reg) = MachineRegister::from_name(target_name) {
                        if reg == MachineRegister::INTO {
                            evaluator.env.write_into(src_val.clone());
                        } else if reg == MachineRegister::DISCARD {
                            // Bit-bucket: safely dropped
                        } else {
                            evaluator.env.set(target_name, src_val.clone());
                        }
                        return Ok(src_val);
                    } else if matches!(*dir.source, ASTNode::Number(_) | ASTNode::StringLit(_) | ASTNode::Ternary(_)) {
                        evaluator.env.set(target_name, src_val.clone());
                        return Ok(src_val);
                    }
                }
            }

            // 4. Evaluate target action
            let target_val = evaluator.eval(&dir.target)?;

            // If target was not an explicit assignment, and source is an accumulator identifier,
            // update source with target result (e.g. A --> A plus new_ball)
            let is_target_an_assignment = matches!(*dir.target, ASTNode::Relation(ref r) if r.relation ==
crate::core::operators::BeingRelation::Is);
            if !is_target_an_assignment {
                if let ASTNode::Identifier(ref name) = *dir.source {
                    evaluator.debugger.assert_mutation_permitted(name).ok();
                    evaluator.env.set(name, target_val.clone());
                }
            }

            Ok(target_val)
```

```
        }

        // ---------------------------------------------------------------------
        // DirectionalVector::ActAgain (<--)
        // ---------------------------------------------------------------------
        DirectionalVector::ActAgain => {
            evaluator.debugger.track_retroaction("ActAgain")?;

            // Indexed Vessel read: Target <-- Vessel[Offset]
            if let ASTNode::Entity(ref target_entity) = *dir.target {
                if let Some(RuntimeValue::Vessel(ref vessel)) = evaluator.env.get(&target_entity.name).cloned() {
                    let offset_val = if let Some(first_el) = target_entity.definition.elements.first() {
                        evaluator.eval(first_el)?
                    } else {
                        RuntimeValue::Number(0.0)
                    };
                    let offset = match offset_val {
                        RuntimeValue::Number(n) => n as usize,
                        _ => 0,
                    };
                    let cell = vessel.read(offset);
                    if let ASTNode::Identifier(ref dest_name) = *dir.source {
                        evaluator.env.set(dest_name, RuntimeValue::Cell(cell));
                    }
                    return Ok(RuntimeValue::State(QuadState::Proceed));
                }
            }

            let target_val = evaluator.eval(&dir.target)?;
            if let RuntimeValue::State(s) = target_val {
                if s == QuadState::Rollback {
                    evaluator.env.rollback_transaction().ok();
                    return Ok(RuntimeValue::State(QuadState::Rollback));
                }
            }
            evaluator.eval(&dir.source)
        }

        // ---------------------------------------------------------------------
        // DirectionalVector::BuildsTo (^--)
        // ---------------------------------------------------------------------
        DirectionalVector::BuildsTo => evaluator.eval(&dir.target),

        // ---------------------------------------------------------------------
        // DirectionalVector::BreaksDownFrom (v--)
        // ---------------------------------------------------------------------
        DirectionalVector::BreaksDownFrom => {
            // Partitioning: ParentStorehouse v-- ChildPartition [ Size ]
            if let ASTNode::Identifier(ref parent_name) = *dir.source {
                if let Some(RuntimeValue::Vessel(mut parent_vessel)) = evaluator.env.get(parent_name).cloned() {
                    if let ASTNode::Entity(ref child_entity) = *dir.target {
                        let cap_val = if let Some(first_el) = child_entity.definition.elements.first() {
                            evaluator.eval(first_el)?
                        } else {
                            RuntimeValue::Number(256.0)
                        };
                        let child_cap = match cap_val {
                            RuntimeValue::Number(n) => n as usize,
                            _ => 256,
                        };
                        match parent_vessel.partition(&child_entity.name, child_cap) {
                            Ok(child_part) => {
                                evaluator.env.set(parent_name, RuntimeValue::Vessel(parent_vessel));
                                evaluator.env.set(&child_entity.name, RuntimeValue::Vessel(child_part));
                                return Ok(RuntimeValue::State(QuadState::Proceed));
                            }
                            Err(err) => {
                                evaluator.debugger.logger.failpoint(FailpointType::BoundaryLeak, "EVALUATOR", &err);
                                return Ok(RuntimeValue::State(QuadState::Rollback));
                            }
                        }
                    }
                }
            }
            evaluator.eval(&dir.target)
        }

        // ---------------------------------------------------------------------
```

```
        // Geometric & Sizing Vectors
        // ------------------------------------------------------------------
        DirectionalVector::Major | DirectionalVector::Minor => evaluator.eval(&dir.target),

        DirectionalVector::Great => evaluator.eval(&dir.source),

        DirectionalVector::Small
        | DirectionalVector::UpperRight
        | DirectionalVector::LowerRight
        | DirectionalVector::UpperLeft
        | DirectionalVector::LowerLeft => evaluator.eval(&dir.target),
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ============================================================================
// Directional vectors advance causal execution without state leakage.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// Directional vector layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/verbs.rs RS 214 LINES 859 WORDS 10.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( SVO Action Verbs, Bijective Transduction & Memory Vessel Allocation )
// ================================================================================
//! OmniCode Runtime: SVO Action Verbs & Vessel Operations
//!
//! # Architectural Scope & Purpose:
//! Evaluates Subject-Verb-Object (SVO) action verbs governing the atomic Character
//! lifecycle:
//! 1. `INSCRIBE`: Outside-in imposition of glyph bytes into 5-trit Cell containers.
//! 2. `TRANSDUCE`: Bijective Radix-3 conversion of 8-bit bytes to 5 balanced trits.
//! 3. `PROJECT`: Projection of Character runes to hardware Target buffers (`INTO` port).
//! 4. `SEQUENCE`: Ingestion of Character runes into the continuous Text ribbon.
//!
//! Also handles physical Vessel and Storehouse memory allocations and indexed accesses.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Epistemic Invariance**: Action verbs drive the physical execution lifecycle.
//!    Documenting each verb ensures consistent behavior across simulated, JIT,
//!    and bare-metal targets.
// ================================================================================

use crate::ast::nodes::{ASTNode, EntityNode};
use crate::core::cell::Cell;
use crate::core::ternary::QuadState;
use crate::core::vessel::Vessel;
use crate::runtime::assembler::MachineRegister;
use crate::runtime::environment::RuntimeValue;
use crate::runtime::evaluator::core::Evaluator;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( SVO Verb Dispatcher Declaration )
// ================================================================================

/// Dispatches SVO action verbs (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`)
/// and Memory Vessel operations. Returns `Ok(Some(value))` if handled, or `Ok(None)`
/// if the entity represents a general ontological definition.
pub fn eval_svo_verb_or_vessel(
    evaluator: &mut Evaluator,
    entity: &EntityNode,
) -> Result<Option<RuntimeValue>, String> {
    // --------------------------------------------------------------------------
    // 1. SVO Action Verbs: INSCRIBE
    // --------------------------------------------------------------------------
    if entity.name == "INSCRIBE" {
        let glyph_byte = if let Some(g_val) = evaluator.env.get("Glyph") {
            if let Some(g) = g_val.character_glyph() {
                g
            } else if let Some(t) = g_val.as_text() {
                t.trim_matches(['\'', '"']).as_bytes().first().copied().unwrap_or(b'A')
            } else {
                b'A'
            }
        } else if let Some(from_val) = evaluator.env.get("%from") {
            if let Some(g) = from_val.character_glyph() {
                g
            } else if let Some(t) = from_val.as_text() {
                t.trim_matches(['\'', '"']).as_bytes().first().copied().unwrap_or(b'A')
            } else {
                b'A'
```

```
            }
        } else {
            b'A'
        };

        let char_val = RuntimeValue::from_glyph(glyph_byte);
        evaluator.env.set("Character`", char_val.clone());
        evaluator.env.set("Character", char_val.clone());
        evaluator.env.registers.cells[MachineRegister::IT.index() as usize] = char_val.clone();
        evaluator.debugger.logger.log(
            "INSCRIBE",
            format!("Inscribed glyph '{}' (0x{:02X}) into Cell", glyph_byte as char, glyph_byte),
        );
        return Ok(Some(char_val));
    }

    // ---------------------------------------------------------------------
    // 2. SVO Action Verbs: TRANSDUCE
    // ---------------------------------------------------------------------
    if entity.name == "TRANSDUCE" {
        let char_opt = evaluator.env.get("Character`").cloned().or_else(|| evaluator.env.get("Character").cloned());
        let cell = if let Some(RuntimeValue::Character { cell, .. }) = char_opt {
            cell
        } else {
            Cell::from_natural(b'A').unwrap_or_else(|_| Cell::from_state(QuadState::Proceed))
        };
        evaluator.env.set("Cell`", RuntimeValue::Cell(cell));
        evaluator.env.set("Cell", RuntimeValue::Cell(cell));
        evaluator.debugger.logger.log(
            "TRANSDUCE",
            format!("Transduced Character byte to 5 balanced trits: {:?}", cell.trits),
        );
        return Ok(Some(RuntimeValue::Cell(cell)));
    }

    // ---------------------------------------------------------------------
    // 3. SVO Action Verbs: PROJECT
    // ---------------------------------------------------------------------
    if entity.name == "PROJECT" {
        let char_opt = evaluator.env.get("Character`").cloned().or_else(|| evaluator.env.get("Character").cloned());
        if let Some(RuntimeValue::Character { glyph, cell }) = char_opt {
            evaluator.env.write_into(RuntimeValue::Character { glyph, cell });
            evaluator.env.set("Target`", RuntimeValue::State(QuadState::Proceed));
            evaluator.debugger.logger.log(
                "PROJECT",
                format!("Projected Character '{}' (0x{:02X}) to Target buffer", glyph as char, glyph),
            );
            return Ok(Some(RuntimeValue::Character { glyph, cell }));
        } else {
            evaluator.env.set("Target`", RuntimeValue::State(QuadState::Proceed));
            return Ok(Some(RuntimeValue::State(QuadState::Proceed)));
        }
    }

    // ---------------------------------------------------------------------
    // 4. SVO Action Verbs: SEQUENCE
    // ---------------------------------------------------------------------
    if entity.name == "SEQUENCE" {
        let char_opt = evaluator.env.get("Character`").cloned().or_else(|| evaluator.env.get("Character").cloned());
        let glyph_char = if let Some(RuntimeValue::Character { glyph, .. }) = char_opt {
            glyph as char
        } else {
            'A'
        };
        let new_text = if let Some(RuntimeValue::Text(ref existing)) = evaluator.env.get("Text").cloned() {
            format!("{}{}", existing, glyph_char)
        } else {
            glyph_char.to_string()
        };
        evaluator.env.set("Text`", RuntimeValue::Text(new_text.clone()));
        evaluator.env.set("Text", RuntimeValue::Text(new_text.clone()));
        evaluator.debugger.logger.log(
            "SEQUENCE",
            format!("Sequenced Character '{}' into Text ribbon: \"{}\"", glyph_char, new_text),
        );
        return Ok(Some(RuntimeValue::Text(new_text)));
    }

```

```
    // ------------------------------------------------------------------------
    // 5. Vessel Indexed Access: Vessel[offset]
    // ------------------------------------------------------------------------
    if let Some(RuntimeValue::Vessel(ref vessel)) = evaluator.env.get(&entity.name).cloned() {
        if let Some(first_el) = entity.definition.elements.first() {
            let offset_val = evaluator.eval(first_el)?;
            let offset = match offset_val {
                RuntimeValue::Number(n) => n as usize,
                _ => 0,
            };
            let cell = vessel.read(offset);
            return Ok(Some(RuntimeValue::Cell(cell)));
        }
    }

    // ------------------------------------------------------------------------
    // 6. Memory Vessel Declaration (Storehouse, Vessel, Partition)
    // ------------------------------------------------------------------------
    let is_vessel_decl = entity.context.is_some()
        || entity.name == "Storehouse"
        || entity.name == "Vessel"
        || entity.name == "Partition"
        || entity.name.ends_with("_Storehouse")
        || entity.name.ends_with("_Partition")
        || entity.name.ends_with("_Vessel")
        || entity.name.starts_with("Storehouse_")
        || entity.name.starts_with("Partition_")
        || entity.name.starts_with("Vessel_")
        || entity.name.starts_with("Main_")
        || entity.name.starts_with("Task_");

    if is_vessel_decl {
        let val = evaluator.eval(&ASTNode::Boundary(entity.definition.clone()))?;
        if let RuntimeValue::Number(cap) = val {
            let is_storehouse = entity.name.contains("Storehouse");
            let vessel = if is_storehouse {
                Vessel::new_storehouse(&entity.name, cap as usize)
            } else {
                Vessel::new(&entity.name, cap as usize)
            };
            evaluator.env.set(&entity.name, RuntimeValue::Vessel(vessel.clone()));
            evaluator.debugger.logger.log("VESSEL", format!("Allocated {vessel}"));
            return Ok(Some(RuntimeValue::Vessel(vessel)));
        }
    }

    // Not an SVO verb or special vessel declaration
    Ok(None)
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Execution Trace Invariants )
// ============================================================================
// SVO verbs operate directly on the environment and register substrates.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
// ============================================================================
// SVO verbs and vessel layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/universal/decoder.rs

RS | 280 LINES | 1156 WORDS | 10.0 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal AST Node Decoding, Trit Extractors & Horner Expansion Invariants )
// ================================================================================
//! OmniCode Runtime: Universal AST Decoders & Trit Transformers
//!
//! # Architectural Scope & Purpose:
//! Provides zero-allocation AST pattern extractors, balanced ternary trit sequence
//! decoders, and Horner power expansion algorithms for bijective numeric transduction.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Bijective Transduction Invariants**: The 5-trit Horner expansion
//!    $\sum_{i=0}^{N-1} t_i \cdot 3^i$ guarantees exact integer parity (e.g., $65 \iff [-1, +1, +1, -1, +1]$)
//!    across the C, Lua, and Rust VMs.
//! 2. **Symbolic Uniformity**: `extract_symbol_name` normalizes heterogeneous AST wrappers
//!    (`Identifier`, `Polarity`, `[ ROLL 1 ]`, `[ Character' ]`) into canonical lookup keys.
// ================================================================================

use crate::ast::nodes::ASTNode;
use crate::core::ternary::QuadState;
use crate::core::operators::Polarity;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Universal Decoder Function Declarations )
// ================================================================================

/// Extracts a symbolic name from a node (e.g. `[ ROLL 1 ]` -> `"ROLL 1"`, `[ Verify ]` -> `"Verify"`).
pub fn extract_symbol_name(node: &ASTNode) -> Option<String> {
    match node {
        ASTNode::Identifier(id) => Some(id.clone()),
        ASTNode::Polarity(p) => Some(p.name.clone()),
        ASTNode::Boundary(b) => {
            if b.elements.len() == 1 {
                extract_symbol_name(&b.elements[0])
            } else if b.elements.len() == 2 {
                if let (ASTNode::Identifier(id), ASTNode::Number(n)) = (&b.elements[0], &b.elements[1]) {
                    Some(format!("{} {}", id, *n as i64))
                } else if let (ASTNode::Identifier(id1), ASTNode::Identifier(id2)) = (&b.elements[0], &b.elements[1]) {
                    Some(format!("{} {}", id1, id2))
                } else {
                    None
                }
            } else {
                None
            }
        }
        ASTNode::Entity(e) => Some(e.name.clone()),
        _ => None,
    }
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Balanced Trit Extraction & Horner Polynomial Evaluation )
// ================================================================================

/// Extracts an array of balanced trits {-1, 0, +1} from a boundary node.
pub fn extract_trits_from_node(node: &ASTNode) -> Option<Vec<i8>> {
    match node {
```

```
        ASTNode::Boundary(b) => {
            let mut trits = Vec::new();
            for el in &b.elements {
                let t = match el {
                    ASTNode::Number(n) => {
                        let i = *n as i8;
                        if i == -1 || i == 0 || i == 1 {
                            i
                        } else {
                            return None;
                        }
                    }
                    ASTNode::Polarity(p) => match p.polarity {
                        Polarity::Abstract => 1,
                        Polarity::Concrete => -1,
                    },
                    ASTNode::Ternary(s) => match s {
                        QuadState::Proceed => 1,
                        QuadState::Await => 0,
                        QuadState::Rollback => -1,
                        QuadState::Halt => 0,
                    },
                    ASTNode::Identifier(id) => match id.as_str() {
                        "-1" => -1,
                        "0" => 0,
                        "+1" | "1" => 1,
                        _ => return None,
                    },
                    _ => return None,
                };
                trits.push(t);
            }
            if !trits.is_empty() {
                Some(trits)
            } else {
                None
            }
        }
        _ => None,
    }
}

/// Computes the balanced integer value of a balanced trit slice using Horner's method:
/// `sum_{i=0..N-1} trits[i] * 3^i`.
pub fn trits_to_integer(trits: &[i8]) -> i64 {
    let mut total: i64 = 0;
    let mut multiplier: i64 = 1;
    for &t in trits {
        total += (t as i64) * multiplier;
        multiplier *= 3;
    }
    total
}

/// Converts an integer into a balanced ternary representation of length
/// `num_trits`.
pub fn integer_to_trits(mut n: i64, num_trits: usize) -> Vec<i8> {
    let mut trits = Vec::with_capacity(num_trits);
    for _ in 0..num_trits {
        let rem = ((n % 3) + 3) % 3;
        match rem {
            0 => {
                trits.push(0);
                n /= 3;
            }
            1 => {
                trits.push(1);
                n = (n - 1) / 3;
            }
            2 => {
                trits.push(-1);
                n = (n + 1) / 3;
            }
            _ => unreachable!(),
        }
    }
    trits
}
```

```rust

/// Converts a balanced trit sequence to dual-rail (pos, neg) bitmasks.
/// For each trit index `i`:
/// - `+1`: bit `i` set in `pos_plane`
/// - `-1`: bit `i` set in `neg_plane`
/// - `0`: bit `i` clear in both planes
pub fn trits_to_dual_rail(trits: &[i8]) -> (u64, u64) {
    let mut pos: u64 = 0;
    let mut neg: u64 = 0;
    for (i, &t) in trits.iter().enumerate().take(64) {
        match t {
            1 => pos |= 1u64 << i,
            -1 => neg |= 1u64 << i,
            _ => {}
        }
    }
    (pos, neg)
}

/// Reconstructs balanced trits from dual-rail bit-planes with collision check.
/// Returns an error if any bit position has both rails asserted
/// (`pos & neg != 0`).
pub fn dual_rail_to_trits(
    pos: u64,
    neg: u64,
    num_trits: usize,
) -> Result<Vec<i8>, String> {
    let collision = pos & neg;
    if collision != 0 {
        return Err(format!(
            "FAILPOINT [DualRailCollision]: Trap at mask 0x{:016X}",
            collision
        ));
    }
    let mut trits = Vec::with_capacity(num_trits);
    for i in 0..num_trits.min(64) {
        let p = (pos >> i) & 1;
        let n = (neg >> i) & 1;
        match (p, n) {
            (0, 0) => trits.push(0),
            (1, 0) => trits.push(1),
            (0, 1) => trits.push(-1),
            _ => unreachable!(),
        }
    }
    Ok(trits)
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Bijective Verification Proofs )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::boundaries::BoundaryType;
    use crate::ast::nodes::BoundaryNode;

    #[test]
    fn test_extract_symbol_name_variants() {
        let id_node = ASTNode::Identifier("Status`".to_string());
        assert_eq!(extract_symbol_name(&id_node), Some("Status`".to_string()));

        let bound_single = ASTNode::Boundary(BoundaryNode {
            boundary_type: BoundaryType::Denotation,
            elements: vec![ASTNode::Identifier("Verify".to_string())],
            aside: None,
            line: 1,
            col: 1,
        });
        assert_eq!(extract_symbol_name(&bound_single), Some("Verify".to_string()));

        let bound_double = ASTNode::Boundary(BoundaryNode {
            boundary_type: BoundaryType::Denotation,
            elements: vec![
                ASTNode::Identifier("ROLL".to_string()),
                ASTNode::Number(1.0),
```

```
            ],
            aside: None,
            line: 1,
            col: 1,
        });
        assert_eq!(extract_symbol_name(&bound_double), Some("ROLL 1".to_string()));
    }

    #[test]
    fn test_bijective_trits_horner_expansion() {
        // 'A' (ASCII 65): [-1, 1, 1, -1, 1]
        // -1*(1) + 1*(3) + 1*(9) - 1*(27) + 1*(81) = -1 + 3 + 9 - 27 + 81 = 65
        let trits_a: [i8; 5] = [-1, 1, 1, -1, 1];
        assert_eq!(trits_to_integer(&trits_a), 65);

        // '0' (ASCII 48): [0, 1, -1, -1, 1]
        // 0*(1) + 1*(3) - 1*(9) - 1*(27) + 1*(81) = 3 - 9 - 27 + 81 = 48
        let trits_0: [i8; 5] = [0, 1, -1, -1, 1];
        assert_eq!(trits_to_integer(&trits_0), 48);
    }

    #[test]
    fn test_integer_to_trits_and_dual_rail_round_trip() {
        // Round trip 65 ('A')
        let trits_65 = integer_to_trits(65, 5);
        assert_eq!(trits_65, vec![-1, 1, 1, -1, 1]);
        assert_eq!(trits_to_integer(&trits_65), 65);

        // Round trip 48 ('0')
        let trits_48 = integer_to_trits(48, 5);
        assert_eq!(trits_48, vec![0, 1, -1, -1, 1]);
        assert_eq!(trits_to_integer(&trits_48), 48);

        // Dual rail bit planes for 'A' [-1, 1, 1, -1, 1]:
        // pos bits: 1, 2, 4 (0b10110 = 22)
        // neg bits: 0, 3 (0b01001 = 9)
        let (pos, neg) = trits_to_dual_rail(&trits_65);
        assert_eq!(pos, 0b10110);
        assert_eq!(neg, 0b01001);
        assert_eq!(pos & neg, 0); // No collision

        let recovered = dual_rail_to_trits(pos, neg, 5).unwrap();
        assert_eq!(recovered, trits_65);

        // Test collision detection
        let collision_res = dual_rail_to_trits(0b11, 0b11, 2);
        assert!(collision_res.is_err());
    }
}

```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/evaluator/universal/mod.rs

RS | 38 LINES | 171 WORDS | 1.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Evaluator Helpers Subsystem Identity & Public Exports )
// ================================================================================
//! OmniCode Runtime: Universal Evaluator Helpers Module
//!
//! Re-exports AST decoders, trit extractors, and Horner expansion functions.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ================================================================================

pub mod decoder;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Universal Decoding Helpers )
// ================================================================================
pub use decoder::{
    extract_symbol_name,
    extract_trits_from_node,
    trits_to_integer,
};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Proofs & Invariants )
// ================================================================================
// Decoders are pure and stateless across all evaluation steps.

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ================================================================================
// Universal evaluation decoder layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/README.md MD 160 LINES 1012 WORDS 8.3 KB

```
# OmniCode Runtime Environment Subsystem

```text
================================================================================
BLOCK 1: METADATA & ARCHITECTURAL MANIFESTO
================================================================================
Subsystem: OmniCode Host Engine & Virtual Machine Runtime Environment
Layer:     Execution Substrate & Transactional State Manager
Path:      <TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/
================================================================================
```

> **"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."**
> — *Exodus 31:18 (KJV)*
> *(The immutable inscription of atomic runes onto bounded physical tablets).*
> **"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."**
> — *Proverbs 24:27 (KJV)*
> *(The discipline of establishing atomic data types and state substrates prior to constructing execution engines).*
> **"Remove not the ancient landmark, which thy fathers have set."**
> — *Proverbs 22:28 (KJV)*
> *(The preservation of historical memory boundaries, silicon addresses, and machine constants).*

---

1. Architectural Mission

The **Runtime Environment Subsystem** is the foundational state and data substrate for the OmniCode Virtual Machine. It decouples **value representation, physical hardware state, and transactional scoping** from the **imperative execution engine (the Evaluator)**.

By treating the environment as an independent, modular library rather than an ad-hoc dictionary:

1. **Any VM subsystem** (Evaluator, Assembler, Diagnostics, REPL, or AI Cortex) can interrogate and construct universal runtime values without circular dependencies.
2. **Reversible Execution** under balanced ternary `-1:ROLLBACK` and speculative guard failures (`! BUT`) is mathematically guaranteed through atomic transactional snapshots.
3. **Cross-Substrate Parity** is achieved across continuous text ribbons, 8-bit glyph bytes, dense 5-trit balanced ternary cells, and structured entity records.

---

2. Topological Layer Structure

The subsystem is organized into three strictly stratified architectural layers:

```bash
src/runtime/environment/
├── universal/                  <-- Tier 1: Universal Runtime Values & Metrics
│   ├── values.rs               <-- RuntimeValue, Unwrappers, Relational Equality, Metrics
│   └── mod.rs                  <-- Re-exports Universal Substrate
│
├── hardware/                   <-- Tier 2: Physical Silicon Register Bank & Ports
│   ├── registers.rs            <-- RegisterBank (13 Named Registers), OMNI_TARGET_*
│   └── mod.rs                  <-- Re-exports Hardware Substrate
│
├── scope/                      <-- Tier 3: Transactional Execution State & Scopes
│   ├── frame.rs                <-- CallFrame (SPOOL Activation Records)
│   ├── snapshot.rs             <-- EnvironmentSnapshot (Full Transactional Image)
│   ├── environment.rs          <-- Environment Coordinator (Bindings, Streams, State-Primes)
│   └── mod.rs                  <-- Re-exports Scope Primitives
│
├── mod.rs                      <-- Subsystem Facade & Backward-Compatible API
└── README.md                   <-- Architectural Manifesto & Specification
```

Layer 1: Universal (`universal/`)

- **`values.rs`**: Contains `RuntimeValue`, the core algebraic data type for all runtime entities:
  - `State(QuadState)`: Balanced ternary truth (+1: Proceed, 0: Await, -1: Rollback, NULL: Halt).
  - `Number(f64)`: 64-bit numeric scalar.
  - `Text(String)`: Continuous character sequence ribbon.
  - `Cell(Cell)`: 5-trit balanced ternary cell spanning 243 discrete states.
  - `Character { glyph: u8, cell: Cell }`: Dual-substrate atomic rune unifying 8-bit ASCII glyph with a 5-trit cell.
```

```
  - `Vessel(Vessel)`: Bounded multi-cell memory vessel.
  - `EntityRecord`: Dynamic record with field map and mutability contract.
  - `Vector`: Ordered sequence of runtime values.
- **Universal Introspection & Coercion**:
  - `as_state()`, `as_number()`, `as_text()`, `as_cell()`, `as_character()`, `as_vector()`.
  - `to_quad_state()`: Universal balanced ternary truth projection.
  - `is_glyph_equal()`: Cross-substrate relational equivalence testing without type friction.
  - `capacity_in_cells()`: Universal memory metric for invariant bound checks (e.g., `{ Character' <= [ 1 Cell ] }`).
- **Dependency Rule**: Zero dependencies on `hardware/` or `scope/`. Pure data substrate.

### Layer 2: Hardware (`hardware/`)

- **`registers.rs`**: Implements the 13 Named Silicon Machine Registers:
  - `%head` (0): Instruction Pointer / Tape Head
  - `%home` (1): Activation Frame Origin
  - `%spool` (2): Stack Spool / Activation Record Floor
  - `%it` (3): Primary Accumulator
  - `%self` (4): Instance Context
  - `%caller` (5): Invoker Context
  - `%other` (6): Peer Context
  - `%state` (7): Active QuadState Status
  - `%turn` (8): Temporal Execution Cadence
  - `%bound` (9): Watchdog Horizon Limit
  - `%from` (10): Inbound Hardware Port / Input Stream
  - `%into` (11): Outbound Hardware Port / Output Stream
  - `%discard` (12): Bit-Bucket Null Sink (writes are dropped, preserving invariance)
- **Canonical Hardware Ports**:
  - `OMNI_TARGET_STDOUT` (`0x01`): Standard terminal console.
  - `OMNI_TARGET_VGA` (`0x000B8000`): Direct video buffer memory address.
  - `OMNI_TARGET_COM1` (`0x03F8`): Serial communications port.
- **Dependency Rule**: Depends only on `universal/values.rs` and `assembler::MachineRegister`.

### Layer 3: Scope (`scope/`)

- **`frame.rs`**: `CallFrame` activation records tracking caller context, base pointers, and local scopes.
- **`snapshot.rs`**: `EnvironmentSnapshot` capturing the complete six-fold transactional horizon:
  1. Symbol table bindings (`bindings`)
  2. Silicon registers (`registers`)
  3. Call stack depth (`call_stack_len`)
  4. Outbound stream length (`output_len`)
  5. Continuous sequence ribbon length (`sequence_len`)
  6. Inbound stream buffer queue (`input_stream`)
- **`environment.rs`**: The state coordinator managing transactions (`begin`, `commit`, `rollback`), state-prime resolution (`Character'` <-> `Character`), procedural frames, and transducer I/O ports.

---

## 3. Reversible Computing & Rollback Guarantees

In OmniCode, computation is inherently reversible. Under balanced ternary logic:

- `+1:PROCEED` commits mutations to the active scope.
- `-1:ROLLBACK` or guard failure (`! BUT`) immediately calls `rollback_transaction()`.

Because `EnvironmentSnapshot` records not only bindings and registers, but also stream truncation horizons (`sequence_ribbon` and `output_stream`) and clones `input_stream`, rolling back a speculative branch leaves **zero side-effects**:

```rust
pub fn rollback_transaction(&mut self) -> Result<(), String> {
    if let Some(prev) = self.snapshots.pop() {
        self.bindings = prev.bindings;
        self.registers = prev.registers;
        self.call_stack.truncate(prev.call_stack_len);
        self.output_stream.truncate(prev.output_len);
        self.sequence_ribbon.truncate(prev.sequence_len);
        self.input_stream = prev.input_stream;
        Ok(())
    } else {
        Err("FAILPOINT [RollbackUnderflow]: No active snapshot".into())
    }
}
```

---

4. State-Prime Resolution Invariant
```

```
Identifiers in OmniCode can be decorated with state-prime affixes (`Character'` or `Character```) indicating temporal progression:

- When looking up `Character'`, if not yet explicitly bound, the environment falls back to `Character`.
- When mutating `Character'`, the environment writes the primed identifier and synchronizes the base `Character` symbol.
- In relational comparisons (`{ Character' = Glyph }`), the evaluator queries `is_glyph_equal` without mutating base state.

---

## 5. Canonical Four-Block Layout

Every source file in the Environment subsystem strictly adheres to the **Canonical 4-Block Structure**:

1. **Block 1: Metadata Block**: File scope, scripture anchors, and AI/developer context rationale.
2. **Block 2: Opening Block**: Core imports, struct/enum definitions, and baseline initializers.
3. **Block 3: Body Block**: Business logic, operations, and trait implementations.
4. **Block 4: Closing Block**: Display formatting, default traits, and verification unit tests.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/mod.rs RS | 76 LINES | 393 WORDS | 3.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Environment Subsystem Facade & Comprehensive Re-exports )
// ============================================================================
//! OmniCode Runtime: Transactional Environment & Register Subsystem
//!
//! # Architectural Scope & Purpose:
//! Modularizes the runtime data substrate and execution state into three
//! decoupled, topologically ordered layers:
//! 1. `universal`: Universal `RuntimeValue` enum, scalar unwrappers, cross-substrate
//!    glyph equality, and physical memory capacity metrics.
//! 2. `hardware`: Physical Silicon `RegisterBank` (the 13 Named Machine Registers)
//!    and canonical hardware device target addresses (`OMNI_TARGET_*`).
//! 3. `scope`: Procedural `CallFrame`, transactional `EnvironmentSnapshot`, and the
//!    `Environment` state coordinator.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Backward Compatibility**: Re-exports all public types at the root of `environment`
//!    ensuring zero downstream breaking changes for the Evaluator, Assembler, CLI,
//!    and Test suites.
//! 2. **Modular Reusability**: Evaluator modules and external crates can import
//!    `universal::RuntimeValue` without pulling in hardware or transaction management.
// ============================================================================

pub mod universal;
pub mod hardware;
pub mod scope;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Full Backward Compatibility )
// ============================================================================

// Universal Value Substrate
pub use universal::RuntimeValue;

// Hardware Silicon Registers & Ports
pub use hardware::{
    RegisterBank,
    OMNI_TARGET_STDOUT,
    OMNI_TARGET_VGA,
    OMNI_TARGET_COM1,
};

// Scopes, Snapshots & Environment State
pub use scope::{
    CallFrame,
    EnvironmentSnapshot,
    Environment,
};

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Structural Proofs & Invariants )
// ============================================================================
// The Environment serves as the foundational data substrate for the Evaluator.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Verification & Module Closure )
```

```
// ================================================================================
// Runtime environment subsystem verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/hardware/mod.rs

RS | 39 LINES | 175 WORDS | 1.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Silicon Hardware Layer Subsystem Identity & Public Exports )
// ================================================================================
//! OmniCode Runtime: Silicon Hardware Substrate Module
//!
//! Provides the physical machine register bank and memory-mapped hardware constants.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ================================================================================

pub mod registers;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Hardware Register Bank & Target Addresses )
// ================================================================================
pub use registers::{
    RegisterBank,
    OMNI_TARGET_STDOUT,
    OMNI_TARGET_VGA,
    OMNI_TARGET_COM1,
};

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Hardware Substrate Invariants & Proofs )
// ================================================================================
// Hardware registers map directly to VM execution state.

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ================================================================================
// Silicon hardware substrate verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/hardware/registers.rs

RS | 166 LINES | 798 WORDS | 7.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Physical Silicon Register Bank & Canonical Hardware Target Ports )
// ================================================================================
//! OmniCode Runtime: Silicon Register Substrate & Hardware Ports
//!
//! # Architectural Scope & Purpose:
//! Implements the 13 Named Physical Silicon Registers of the OmniCode Host Engine:
//! - `%head`    (0): Tape Head / Instruction Pointer
//! - `%home`    (1): Base Pointer / Frame Origin
//! - `%spool`   (2): Stack Spool / Activation Record Floor
//! - `%it`      (3): Primary Accumulator / Result
//! - `%self`    (4): Current Instance / Context Entity
//! - `%caller`  (5): Invoker Context Entity
//! - `%other`   (6): Peer / Auxiliary Entity
//! - `%state`   (7): Active QuadState Execution Status
//! - `%turn`    (8): Temporal Cadence / Cycle Counter
//! - `%bound`   (9): Watchdog Horizon / Recursion Limit
//! - `%from`   (10): Inbound Hardware Port / Input Stream
//! - `%into`   (11): Outbound Hardware Port / Output Stream
//! - `%discard`(12): Bit-Bucket / Null Sink (writes are safely dropped)
//!
//! Also defines canonical memory-mapped device target addresses (`OMNI_TARGET_*`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Silicon Parity**: Autonomous coding agents must respect that `%discard`
//!    is an immutable sink; writes to it MUST NEVER mutate state or cause panics.
//! 2. **Physical Addresses**: Canonical constants ensure hardware device ports
//!    remain stable across host architecture revisions.
// ================================================================================

use crate::core::ternary::QuadState;
use crate::runtime::assembler::MachineRegister;
use crate::runtime::environment::universal::RuntimeValue;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Canonical Hardware Addresses & RegisterBank Definition )
// ================================================================================

/// Canonical Hardware Target Addresses
pub const OMNI_TARGET_STDOUT: u64 = 0x01;
pub const OMNI_TARGET_VGA: u64 = 0x000B8000;
pub const OMNI_TARGET_COM1: u64 = 0x03F8;

/// Physical Silicon Register Bank hosting the 13 Named Machine Registers.
///
/// Implements immediate hardware cell reading, mutation, and discard-sink semantics.
#[derive(Debug, Clone, PartialEq)]
pub struct RegisterBank {
    pub cells: [RuntimeValue; 13],
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Register Accessors, Mutation, Discard-Sink Semantics & Name Lookup )
// ================================================================================

impl RegisterBank {
    /// Initializes the silicon register bank to pristine hardware baselines.
```

```
    pub fn new() -> Self {
        Self {
            cells: [
                RuntimeValue::Number(0.0),                  // 0: HEAD (Tape Head / IP)
                RuntimeValue::Number(0.0),                  // 1: HOME (Base Pointer / Origin)
                RuntimeValue::Number(0.0),                  // 2: SPOOL (Stack Spool / Floor)
                RuntimeValue::State(QuadState::Proceed),    // 3: IT (Accumulator / Result)
                RuntimeValue::Text("Kernel".to_string()),   // 4: SELF (Instance / Context)
                RuntimeValue::Text("Root".to_string()),     // 5: CALLER (Invoker Context)
                RuntimeValue::Text("Swarm".to_string()),    // 6: OTHER (Peer / Auxiliary)
                RuntimeValue::State(QuadState::Proceed),    // 7: STATE (QuadState Status)
                RuntimeValue::Number(0.0),                  // 8: TURN (Temporal Cadence / Cycle)
                RuntimeValue::Number(0.0),                  // 9: BOUND (Watchdog Limit / Horizon)
                RuntimeValue::Text(String::new()),          // 10: FROM (Input Channel / Stdin)
                RuntimeValue::Text(String::new()),          // 11: INTO (Output Channel / Stdout)
                RuntimeValue::State(QuadState::Proceed),    // 12: DISCARD (Bit-Bucket / Null Sink)
            ],
        }
    }

    /// Reads a register cell by its physical `MachineRegister` enum.
    pub fn read(&self, reg: MachineRegister) -> RuntimeValue {
        self.cells[reg.index() as usize].clone()
    }

    /// Writes a value to a register cell. Writes to `DISCARD` are safely dropped.
    pub fn write(&mut self, reg: MachineRegister, val: RuntimeValue) {
        if reg == MachineRegister::DISCARD {
            return;
        }
        self.cells[reg.index() as usize] = val;
    }

    /// Reads a register by name if recognized, returning `None` otherwise.
    pub fn read_by_name(&self, name: &str) -> Option<RuntimeValue> {
        MachineRegister::from_name(name).map(|r| self.read(r))
    }

    /// Writes to a register by name. Returns true if name was a recognized register.
    pub fn write_by_name(&mut self, name: &str, val: RuntimeValue) -> bool {
        if let Some(reg) = MachineRegister::from_name(name) {
            self.write(reg, val);
            true
        } else {
            false
        }
    }

    /// Resets all registers to pristine hardware baseline.
    pub fn reset(&mut self) {
        *self = Self::new();
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation & Unit Tests )
// ============================================================================

impl Default for RegisterBank {
    fn default() -> Self {
        Self::new()
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_register_bank_initial_state() {
        let bank = RegisterBank::new();
        assert_eq!(bank.read(MachineRegister::HEAD), RuntimeValue::Number(0.0));
        assert_eq!(bank.read(MachineRegister::IT), RuntimeValue::State(QuadState::Proceed));
        assert_eq!(bank.read(MachineRegister::SELF), RuntimeValue::Text("Kernel".to_string()));
    }

    #[test]
```

```
    fn test_register_discard_sink_invariance() {
        let mut bank = RegisterBank::new();
        bank.write(MachineRegister::DISCARD, RuntimeValue::Number(9999.0));
        assert_eq!(bank.read(MachineRegister::DISCARD), RuntimeValue::State(QuadState::Proceed));
    }

    #[test]
    fn test_register_read_write_by_name() {
        let mut bank = RegisterBank::new();
        assert!(bank.write_by_name("IT", RuntimeValue::Number(42.0)));
        assert_eq!(bank.read_by_name("IT"), Some(RuntimeValue::Number(42.0)));
        assert!(!bank.write_by_name("NON_EXISTENT_REGISTER", RuntimeValue::Number(1.0)));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/environment.rs

RS | 291 LINES | 1098 WORDS | 12.2 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Runtime Environment State, Transactional Scopes & Symbol Table Invariants )
// ================================================================================
//! OmniCode Runtime: Transactional Execution Environment
//!
//! # Architectural Scope & Purpose:
//! The `Environment` orchestrates scoped ontological symbol tables with native snapshot,
//! transaction nesting, and zero-cost rollback capabilities to support balanced
//! ternary -1 (Rollback) state transitions.
//!
//! In addition to lexical variable bindings, the Environment coordinates the physical
//! Silicon Register Bank (the 13 Named Machine Registers), procedural call frames,
//! hardware transducer ports (`FROM`, `INTO`), continuous text ribbons (`sequence_ribbon`),
//! and state-prime symbol resolution.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Epistemic Continuity**: Autonomous coding agents require explicit rationale
//!    for why physical registers are synchronized with symbol bindings and why
//!    state-primes (`Character'`) automatically resolve to base entities (`Character`).
//! 2. **Transactional Soundness**: Full restoration of `bindings`, `registers`, `call_stack`,
//!    `output_stream`, `sequence_ribbon`, and `input_stream` ensures zero state leakage
//!    during reversible execution branches.
// ================================================================================

use std::collections::HashMap;
use crate::runtime::assembler::MachineRegister;
use crate::runtime::environment::universal::RuntimeValue;
use crate::runtime::environment::hardware::{
    RegisterBank,
    OMNI_TARGET_STDOUT,
};
use crate::runtime::environment::scope::frame::CallFrame;
use crate::runtime::environment::scope::snapshot::EnvironmentSnapshot;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Environment Struct Definition & Initialization )
// ================================================================================

/// Transactional Runtime Environment managing symbol tables, registers, and execution streams.
#[derive(Debug, Clone)]
pub struct Environment {
    pub registers: RegisterBank,
    bindings: HashMap<String, RuntimeValue>,
    call_stack: Vec<CallFrame>,
    pub output_stream: Vec<RuntimeValue>,
    pub input_stream: Vec<RuntimeValue>,
    pub target_device: u64,
    pub sequence_ribbon: Vec<RuntimeValue>,
    snapshots: Vec<EnvironmentSnapshot>,
}

impl Environment {
    /// Creates a pristine runtime environment with default hardware registers.
    pub fn new() -> Self {
        Self {
            registers: RegisterBank::new(),
            bindings: HashMap::new(),
            call_stack: Vec::new(),
```

```
            output_stream: Vec::new(),
            input_stream: Vec::new(),
            target_device: OMNI_TARGET_STDOUT,
            sequence_ribbon: Vec::new(),
            snapshots: Vec::new(),
        }
    }
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Transactional Operations, Binding Mutations, Call Stacks & Transducer I/O )
// ================================================================================

impl Environment {
    /// Takes an atomic transactional snapshot of current bindings, registers, and streams.
    pub fn begin_transaction(&mut self) {
        self.snapshots.push(EnvironmentSnapshot::capture(
            self.bindings.clone(),
            self.registers.clone(),
            self.call_stack.len(),
            self.output_stream.len(),
            self.sequence_ribbon.len(),
            self.input_stream.clone(),
        ));
    }

    /// Commits the active transaction, discarding the restore snapshot.
    pub fn commit_transaction(&mut self) {
        self.snapshots.pop();
    }

    /// Unwinds and restores the environment state completely to the last snapshot.
    pub fn rollback_transaction(&mut self) -> Result<(), String> {
        if let Some(prev) = self.snapshots.pop() {
            self.bindings = prev.bindings;
            self.registers = prev.registers;
            self.call_stack.truncate(prev.call_stack_len);
            self.output_stream.truncate(prev.output_len);
            self.sequence_ribbon.truncate(prev.sequence_len);
            self.input_stream = prev.input_stream;
            Ok(())
        } else {
            Err("FAILPOINT [RollbackUnderflow]: Attempted rollback with no active transaction snapshot".into())
        }
    }

    /// Returns a reference to the active symbol bindings table.
    pub fn bindings(&self) -> &HashMap<String, RuntimeValue> {
        &self.bindings
    }

    /// Clears all bindings, call stacks, streams, and resets registers.
    pub fn clear(&mut self) {
        self.bindings.clear();
        self.registers.reset();
        self.call_stack.clear();
        self.output_stream.clear();
        self.input_stream.clear();
        self.target_device = OMNI_TARGET_STDOUT;
        self.sequence_ribbon.clear();
        self.snapshots.clear();
    }

    /// Binds a value by name. If the name corresponds to a physical register,
    /// updates the silicon register cell in addition to the symbol table.
    /// If the name contains state-prime affixes (e.g., Character` or Character'),
    /// it binds the primed name and automatically synchronizes the base entity.
    pub fn set(&mut self, name: &str, value: RuntimeValue) {
        if let Some(reg) = MachineRegister::from_name(name) {
            self.registers.write(reg, value.clone());
            if reg != MachineRegister::DISCARD {
                self.bindings.insert(name.to_string(), value);
            }
        } else {
            self.bindings.insert(name.to_string(), value.clone());
            let trimmed = name.trim_end_matches(['`', '\'']);
            if trimmed != name && !trimmed.is_empty() {
```

```
                self.bindings.insert(trimmed.to_string(), value);
            }
        }
    }

    /// Retrieves a value by name. If the name matches a silicon register,
    /// yields a reference to the physical register cell.
    /// Supports state-prime fallback: looking up Character` falls back to Character
    /// if the specific prime has not yet been bound.
    pub fn get(&self, name: &str) -> Option<&RuntimeValue> {
        if let Some(reg) = MachineRegister::from_name(name) {
            Some(&self.registers.cells[reg.index() as usize])
        } else if let Some(val) = self.bindings.get(name) {
            Some(val)
        } else {
            let trimmed = name.trim_end_matches(['`', '\'']);
            if trimmed != name && !trimmed.is_empty() {
                self.bindings.get(trimmed)
            } else {
                None
            }
        }
    }

    /// Returns true if the environment contains a binding or register for the given name.
    pub fn contains(&self, name: &str) -> bool {
        self.get(name).is_some()
    }

    /// Pushes a procedural call frame onto the `%spool` stack.
    pub fn push_call_frame(&mut self, caller: RuntimeValue, self_val: RuntimeValue, return_head: usize) {
        let home = self.call_stack.len();
        self.registers.write(MachineRegister::HOME, RuntimeValue::Number(home as f64));
        self.registers.write(MachineRegister::HEAD, RuntimeValue::Number(return_head as f64));
        self.registers.write(MachineRegister::CALLER, caller.clone());
        self.registers.write(MachineRegister::SELF, self_val.clone());
        self.call_stack.push(CallFrame::new(caller, self_val, home, return_head));
        self.registers.write(MachineRegister::SPOOL, RuntimeValue::Number(self.call_stack.len() as f64));
    }

    /// Pops the active call frame from the `%spool` stack, restoring caller context.
    pub fn pop_call_frame(&mut self) -> Option<CallFrame> {
        let popped = self.call_stack.pop()?;
        let current_spool = self.call_stack.len();
        self.registers.write(MachineRegister::SPOOL, RuntimeValue::Number(current_spool as f64));
        if let Some(prev) = self.call_stack.last() {
            self.registers.write(MachineRegister::HOME, RuntimeValue::Number(prev.home as f64));
            self.registers.write(MachineRegister::HEAD, RuntimeValue::Number(prev.head as f64));
            self.registers.write(MachineRegister::CALLER, prev.caller.clone());
            self.registers.write(MachineRegister::SELF, prev.self_val.clone());
        }
        Some(popped)
    }

    /// Emits a runtime value to the `INTO` outbound port and buffers it in output_stream.
    pub fn write_into(&mut self, val: RuntimeValue) {
        self.registers.write(MachineRegister::INTO, val.clone());
        self.output_stream.push(val);
    }

    /// Reads the next input value from `input_stream` into `FROM`, or reads current `FROM`.
    pub fn read_from(&mut self) -> RuntimeValue {
        if !self.input_stream.is_empty() {
            let next_val = self.input_stream.remove(0);
            self.registers.write(MachineRegister::FROM, next_val.clone());
            next_val
        } else {
            self.registers.read(MachineRegister::FROM)
        }
    }

    /// Reads the current value held in the `INTO` output register.
    pub fn read_into(&self) -> RuntimeValue {
        self.registers.read(MachineRegister::INTO)
    }

    /// Enqueues an input value into `input_stream` and sets the `FROM` register.
    pub fn write_from(&mut self, val: RuntimeValue) {
```

```
        self.registers.write(MachineRegister::FROM, val.clone());
        self.input_stream.push(val);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Default Trait Implementation & Unit Tests )
// ============================================================================

impl Default for Environment {
    fn default() -> Self {
        Self::new()
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::core::ternary::QuadState;

    #[test]
    fn test_environment_registers_and_discard_sink() {
        let mut env = Environment::new();
        env.set("INTO", RuntimeValue::from_glyph(b'Z'));
        assert_eq!(env.registers.read(MachineRegister::INTO).character_glyph(), Some(b'Z'));

        env.set("DISCARD", RuntimeValue::Number(999.0));
        assert_eq!(env.registers.read(MachineRegister::DISCARD), RuntimeValue::State(QuadState::Proceed));
    }

    #[test]
    fn test_environment_transactional_snapshots_and_rollback() {
        let mut env = Environment::new();
        env.set("A", RuntimeValue::Number(10.0));
        env.sequence_ribbon.push(RuntimeValue::from_char('a'));
        env.write_from(RuntimeValue::from_char('x'));

        env.begin_transaction();
        env.set("A", RuntimeValue::Number(20.0));
        env.sequence_ribbon.push(RuntimeValue::from_char('b'));
        let _ = env.read_from(); // consume 'x'
        assert_eq!(env.get("A"), Some(&RuntimeValue::Number(20.0)));
        assert_eq!(env.sequence_ribbon.len(), 2);
        assert_eq!(env.input_stream.len(), 0);

        env.rollback_transaction().expect("Rollback");
        assert_eq!(env.get("A"), Some(&RuntimeValue::Number(10.0)));
        assert_eq!(env.sequence_ribbon.len(), 1);
        assert_eq!(env.input_stream.len(), 1); // input queue restored!
    }

    #[test]
    fn test_environment_state_prime_synchronization() {
        let mut env = Environment::new();
        env.set("Character`", RuntimeValue::from_char('A'));
        assert!(env.contains("Character`"));
        assert!(env.contains("Character"));
        assert_eq!(env.get("Character"), env.get("Character`"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/frame.rs

RS | 95 LINES | 421 WORDS | 3.7 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Procedural Call Frame & Activation Record Invariants )
// ================================================================================
//! OmniCode Runtime: Procedural Call Frames
//!
//! # Architectural Scope & Purpose:
//! Defines the `CallFrame` activation record preserved on the `%spool` stack
//! during procedural and functional invocation.
//!
//! Preserves the caller entity context, instance context (`self`), return instruction
//! pointer (`head`), base frame pointer (`home`), and local lexical symbol bindings.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Stack Unwinding Safety**: Activation records must unambiguously capture
//!    caller contexts to prevent memory clobbering across reentrant invocations.
// ================================================================================

use std::collections::HashMap;
use crate::runtime::environment::universal::RuntimeValue;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( CallFrame Struct Definition & Field Declarations )
// ================================================================================

/// Call stack frame preserved on `%spool` for procedural execution.
#[derive(Debug, Clone, PartialEq)]
pub struct CallFrame {
    /// The identity or value of the invoking caller.
    pub caller: RuntimeValue,
    /// The instance context (`self`) of the active procedure.
    pub self_val: RuntimeValue,
    /// The base stack depth (`%home`) prior to activation.
    pub home: usize,
    /// The return instruction address (`%head`).
    pub head: usize,
    /// Local scoped symbol bindings specific to this activation frame.
    pub locals: HashMap<String, RuntimeValue>,
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( CallFrame Initialization & Scope Accessors )
// ================================================================================

impl CallFrame {
    /// Creates a new activation record with empty local bindings.
    pub fn new(caller: RuntimeValue, self_val: RuntimeValue, home: usize, head: usize) -> Self {
        Self {
            caller,
            self_val,
            home,
            head,
            locals: HashMap::new(),
        }
    }
}

// ================================================================================
```

```
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proofs )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_call_frame_creation() {
        let frame = CallFrame::new(
            RuntimeValue::Text("Caller".to_string()),
            RuntimeValue::Text("Self".to_string()),
            0,
            100,
        );
        assert_eq!(frame.home, 0);
        assert_eq!(frame.head, 100);
        assert!(frame.locals.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/mod.rs

RS | 39 LINES | 180 WORDS | 1.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Scope & Transaction Layer Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Runtime: Scope & Transaction Management Module
//!
//! Provides procedural call frames, transactional snapshots, and the
//! environment state coordinator.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod frame;
pub mod snapshot;
pub mod environment;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Scope Types )
// ============================================================================
pub use frame::CallFrame;
pub use snapshot::EnvironmentSnapshot;
pub use environment::Environment;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Scope Layer Invariants & Proofs )
// ============================================================================
// Execution environments manage transactional scopes and symbols.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ============================================================================
// Scope layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/scope/snapshot.rs

RS | 109 LINES | 423 WORDS | 4.1 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Transactional Snapshot Invariants & Reversible Execution State )
// ================================================================================
//! OmniCode Runtime: Transactional Environment Snapshot
//!
//! # Architectural Scope & Purpose:
//! Encapsulates an atomic image of the runtime environment at a transaction boundary.
//!
//! Captures:
//! 1. Lexical symbol table bindings (`bindings`).
//! 2. Physical silicon register bank (`registers`).
//! 3. Procedural call stack depth (`call_stack_len`).
//! 4. Hardware outbound stream length (`output_len`).
//! 5. Sequential character ribbon length (`sequence_len`).
//! 6. Exact inbound hardware stream buffer (`input_stream`).
//!
//! Ensures 100% reversible computation under balanced ternary `-1:ROLLBACK`
//! and speculative guard failure (`! BUT`).
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Complete Reversibility**: Truncating `sequence_ribbon` and restoring
//!    dequeued `input_stream` items prevents side-effect leakage during speculative roll evaluations.
// ================================================================================

use std::collections::HashMap;
use crate::runtime::environment::universal::RuntimeValue;
use crate::runtime::environment::hardware::RegisterBank;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( EnvironmentSnapshot Struct Definition )
// ================================================================================

/// Transactional snapshot capturing symbolic bindings, physical registers, and stream horizons.
#[derive(Debug, Clone)]
pub struct EnvironmentSnapshot {
    pub bindings: HashMap<String, RuntimeValue>,
    pub registers: RegisterBank,
    pub call_stack_len: usize,
    pub output_len: usize,
    pub sequence_len: usize,
    pub input_stream: Vec<RuntimeValue>,
}

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Snapshot Constructor & State Invariants )
// ================================================================================

impl EnvironmentSnapshot {
    /// Captures a complete atomic snapshot of the execution state.
    pub fn capture(
        bindings: HashMap<String, RuntimeValue>,
        registers: RegisterBank,
        call_stack_len: usize,
        output_len: usize,
        sequence_len: usize,
        input_stream: Vec<RuntimeValue>,
    ) -> Self {
```

```
        Self {
            bindings,
            registers,
            call_stack_len,
            output_len,
            sequence_len,
            input_stream,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests & Verification Proofs )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_snapshot_capture() {
        let snap = EnvironmentSnapshot::capture(
            HashMap::new(),
            RegisterBank::new(),
            0,
            0,
            0,
            Vec::new(),
        );
        assert_eq!(snap.call_stack_len, 0);
        assert_eq!(snap.sequence_len, 0);
        assert!(snap.input_stream.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/universal/mod.rs

RS | 34 LINES | 167 WORDS | 1.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Value Layer Subsystem Identity & Public Exports )
// ============================================================================
//! OmniCode Runtime: Universal Value Layer Module
//!
//! Provides the foundational, universal value primitives for the OmniCode VM.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**: Tables of testimony, written with the finger of God.
//! - **Proverbs 24:27 (KJV)**: Prepare thy work without, and make it fit for thyself in the field.
//! - **Proverbs 22:28 (KJV)**: Remove not the ancient landmark, which thy fathers have set.
// ============================================================================

pub mod values;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Re-export Declarations for Universal Runtime Values )
// ============================================================================
pub use values::RuntimeValue;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal Invariants & Subsystem Proofs )
// ============================================================================
// Values are pure and decouple representation from execution.

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status & Module Closure )
// ============================================================================
// Universal value layer verified and closed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/runtime/environment/universal/values.rs

RS | 596 LINES | 2210 WORDS | 23.6 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Runtime Value Representation, Unwrapping & Metric Invariants )
// ================================================================================
//! OmniCode Runtime: Universal Runtime Value Substrate
//!
//! # Architectural Scope & Purpose:
//! Defines the universal `RuntimeValue` enum representing every discrete unit of
//! data flowing through the OmniCode Virtual Machine.
//!
//! Provides zero-cost scalar unwrappers, bijective character-cell transducers,
//! balanced ternary quad-state truth projections, cross-substrate relational
//! equality guards, and physical cell capacity metrics.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of atomic runes onto bounded physical tablets).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field;
//!    and afterwards build thine house."
//!   (The discipline of proving atomic types and substrates before constructing systems).
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The preservation of historical memory boundaries, silicon addresses, and constants).
//!
//! # Why Extensive Commenting Matters for AI Agents & Developers:
//! 1. **Cross-Substrate Soundness**: `Character` unifies an 8-bit glyph byte with
//!    a 5-trit balanced ternary `Cell`. Unification methods like `is_glyph_equal`
//!    allow relational algebra to operate seamlessly across text, cells, and characters.
//! 2. **Universal Introspection**: Evaluator sub-modules and execution pipelines
//!    require standardized unwrapping (`as_state`, `as_number`, `as_text`) without
//!    resorting to repeated error-prone match blocks.
//! 3. **Bounded Capacity Invariants**: Relational constraints like `{ Character' <= [ 1 Cell ] }`
//!    require an objective metric `capacity_in_cells()` to verify memory bounds.
// ================================================================================

use std::collections::HashMap;
use std::fmt;
use crate::core::ternary::QuadState;
use crate::core::cell::Cell;
use crate::core::vessel::Vessel;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( RuntimeValue Enum Definition & Core Constructors )
// ================================================================================

/// Represents any discrete value flowing through the OmniCode runtime.
///
/// Implements full dual-substrate parity across balanced ternary truth states,
/// continuous text strings, dense 5-trit cells, and the atomic Character rune.
#[derive(Debug, Clone, PartialEq)]
pub enum RuntimeValue {
    /// Quad-State truth value (+1: Proceed, 0: Await, -1: Rollback, NULL: Halt).
    State(QuadState),
    /// 64-bit floating-point numeric scalar or integer codepoint.
    Number(f64),
    /// Continuous text string ribbon composed of sequential characters.
    Text(String),
    /// Atomic 5-trit balanced ternary memory cell (spanning 243 states).
    Cell(Cell),
    /// Atomic Character rune enclosing both 8-bit ASCII glyph and 5-trit Cell.
    /// Fulfills invariant: `{ Character } = [ Enclosed ]: TRUE`.
    Character {
        glyph: u8,
        cell: Cell,
    },
    /// Bounded multi-cell memory vessel or storehouse partition.
    Vessel(Vessel),
    /// Structured entity record containing named fields and mutability contract.
    EntityRecord {
```

```
        name: String,
        is_immutable: bool,
        values: HashMap<String, Box<RuntimeValue>>,
    },
    /// Ordered vector of runtime values.
    Vector(Vec<RuntimeValue>),
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Constructors, Universal Unwrapping, Relational Helpers & Capacity Metrics )
// ============================================================================

impl RuntimeValue {
    // ------------------------------------------------------------------------
    // Constructors
    // ------------------------------------------------------------------------

    /// Constructs a `RuntimeValue::State` from a balanced ternary `QuadState`.
    pub fn from_state(state: QuadState) -> Self {
        RuntimeValue::State(state)
    }

    /// Constructs a `RuntimeValue::Number` from an `f64`.
    pub fn from_number(n: f64) -> Self {
        RuntimeValue::Number(n)
    }

    /// Constructs a `RuntimeValue::Text` from any string-like type.
    pub fn from_text(t: impl Into<String>) -> Self {
        RuntimeValue::Text(t.into())
    }

    /// Constructs a `RuntimeValue::Cell` from a 5-trit `Cell`.
    pub fn from_cell(cell: Cell) -> Self {
        RuntimeValue::Cell(cell)
    }

    /// Constructs a `RuntimeValue::Character` from an 8-bit ASCII byte/glyph,
    /// bijectively transducing the byte into a 5-trit balanced ternary `Cell`.
    pub fn from_glyph(glyph: u8) -> Self {
        let cell = Cell::from_natural(glyph).unwrap_or_else(|_| Cell::from_state(QuadState::Rollback));
        RuntimeValue::Character { glyph, cell }
    }

    /// Constructs a `RuntimeValue::Character` from a standard Rust character.
    pub fn from_char(c: char) -> Self {
        let glyph = (c as u32 & 0xFF) as u8;
        Self::from_glyph(glyph)
    }

    // ------------------------------------------------------------------------
    // Universal Scalar Extractors & Unwrappers
    // ------------------------------------------------------------------------

    /// Returns the wrapped `QuadState` if this value is a `State`.
    pub fn as_state(&self) -> Option<QuadState> {
        match self {
            RuntimeValue::State(s) => Some(*s),
            _ => None,
        }
    }

    /// Returns the wrapped `f64` if this value is a `Number`.
    pub fn as_number(&self) -> Option<f64> {
        match self {
            RuntimeValue::Number(n) => Some(*n),
            _ => None,
        }
    }

    /// Returns a string slice if this value is `Text`.
    pub fn as_text(&self) -> Option<&str> {
        match self {
            RuntimeValue::Text(t) => Some(t.as_str()),
            _ => None,
        }
    }
```

```rust

    /// Returns the wrapped `Cell` if this value is a `Cell` or extracts the cell substrate of a `Character`.
    pub fn as_cell(&self) -> Option<Cell> {
        match self {
            RuntimeValue::Cell(c) => Some(*c),
            RuntimeValue::Character { cell, .. } => Some(*cell),
            _ => None,
        }
    }

    /// Returns the 8-bit glyph byte and 5-trit cell if this value is a `Character`.
    pub fn as_character(&self) -> Option<(u8, Cell)> {
        match self {
            RuntimeValue::Character { glyph, cell } => Some((*glyph, *cell)),
            _ => None,
        }
    }

    /// Returns a slice of elements if this value is a `Vector`.
    pub fn as_vector(&self) -> Option<&[RuntimeValue]> {
        match self {
            RuntimeValue::Vector(v) => Some(v.as_slice()),
            _ => None,
        }
    }

    /// Returns the 8-bit glyph byte if this value is a Character or convertible numeric/text value.
    pub fn character_glyph(&self) -> Option<u8> {
        match self {
            RuntimeValue::Character { glyph, .. } => Some(*glyph),
            RuntimeValue::Number(n) => {
                if *n >= 0.0 && *n <= 255.0 {
                    Some(*n as u8)
                } else {
                    None
                }
            }
            RuntimeValue::Text(t) => {
                let bytes = t.as_bytes();
                if bytes.len() == 1 {
                    Some(bytes[0])
                } else {
                    None
                }
            }
            _ => None,
        }
    }

    /// Returns the 5-trit `Cell` substrate if this value is a Character or Cell.
    pub fn character_cell(&self) -> Option<Cell> {
        match self {
            RuntimeValue::Character { cell, .. } => Some(*cell),
            RuntimeValue::Cell(c) => Some(*c),
            RuntimeValue::Number(n) => {
                if *n >= 0.0 && *n <= 242.0 {
                    Cell::from_natural(*n as u8).ok()
                } else {
                    None
                }
            }
            _ => None,
        }
    }

    // ---------------------------------------------------------------------------
    // Quad-State Truthiness Predicates
    // ---------------------------------------------------------------------------

    /// Returns true if this value is strictly `+1:PROCEED`.
    pub fn is_proceed(&self) -> bool {
        matches!(self, RuntimeValue::State(QuadState::Proceed))
    }

    /// Returns true if this value is strictly `0:AWAIT`.
    pub fn is_await(&self) -> bool {
        matches!(self, RuntimeValue::State(QuadState::Await))
    }
```

```
    /// Returns true if this value is strictly `-1:ROLLBACK`.
    pub fn is_rollback(&self) -> bool {
        matches!(self, RuntimeValue::State(QuadState::Rollback))
    }

    /// Returns true if this value is strictly `NULL:HALT`.
    pub fn is_halt(&self) -> bool {
        matches!(self, RuntimeValue::State(QuadState::Halt))
    }

    /// Universally projects any runtime value to a balanced ternary `QuadState`.
    pub fn to_quad_state(&self) -> QuadState {
        match self {
            RuntimeValue::State(s) => *s,
            RuntimeValue::Number(n) => {
                if *n > 0.0 {
                    QuadState::Proceed
                } else if *n == 0.0 {
                    QuadState::Await
                } else {
                    QuadState::Rollback
                }
            }
            RuntimeValue::Text(t) => {
                if t.is_empty() {
                    QuadState::Await
                } else {
                    QuadState::Proceed
                }
            }
            RuntimeValue::Cell(c) => c.state,
            RuntimeValue::Character { cell, .. } => cell.state,
            RuntimeValue::Vessel(_) => QuadState::Proceed,
            RuntimeValue::EntityRecord { .. } => QuadState::Proceed,
            RuntimeValue::Vector(v) => {
                if v.is_empty() {
                    QuadState::Await
                } else {
                    QuadState::Proceed
                }
            }
        }
    }

    // ------------------------------------------------------------------------
    // Universal Dual-Substrate Relational Equality
    // ------------------------------------------------------------------------

    /// Compares two runtime values for semantic / glyph equivalence across representations.
    ///
    /// For example:
    /// - `Character('A')` equals `Character('A')`.
    /// - `Character('A')` equals `Text("A")`.
    /// - `Character('A')` equals `Number(65.0)`.
    /// - In general, identical types match by direct value equality.
    pub fn is_glyph_equal(&self, other: &Self) -> bool {
        if self == other {
            return true;
        }

        // Cross-compare glyph representations
        if let (Some(g1), Some(g2)) = (self.character_glyph(), other.character_glyph()) {
            if g1 == g2 {
                return true;
            }
        }

        // Cross-compare cell representations
        if let (Some(c1), Some(c2)) = (self.character_cell(), other.character_cell()) {
            if c1 == c2 {
                return true;
            }
        }

        // Numeric equivalence
        if let (Some(n1), Some(n2)) = (self.as_number(), other.as_number()) {
            return (n1 - n2).abs() < f64::EPSILON;
```

```
        }

        // String equivalence with boundary cast and quote stripping
        if let (Some(t1), Some(t2)) = (self.as_text(), other.as_text()) {
            if t1 == t2 {
                return true;
            }
            let c1 = t1.trim()
                .trim_matches(['\'', '"', '[', ']', '{', '}', '<', '>'])
                .trim();
            let c2 = t2.trim()
                .trim_matches(['\'', '"', '[', ']', '{', '}', '<', '>'])
                .trim();
            if c1 == c2 || c1.eq_ignore_ascii_case(c2) {
                return true;
            }
        }

        // Single-element vector unwrap for boundary-enclosed scalar comparisons
        if let RuntimeValue::Vector(v) = self {
            if v.len() == 1 {
                return v[0].is_glyph_equal(other);
            }
        }
        if let RuntimeValue::Vector(v) = other {
            if v.len() == 1 {
                return self.is_glyph_equal(&v[0]);
            }
        }

        // State vs Number / Text duality (Duality: words & values / states & numbers)
        // Freestanding language semantics:
        // - Literal operator "" ("0", "1", "-1") clarifies literal scalar value
        // - Boundary casts [ 0 ], { 0 }, < 0 > clarify boundary enclosure
        // - Keyword literal '' ('AWAIT', 'PROCEED') clarifies keyword as word
        match (self, other) {
            (RuntimeValue::State(s), RuntimeValue::Number(n))
            | (RuntimeValue::Number(n), RuntimeValue::State(s)) => {
                if let Some(i) = s.as_int() {
                    return (*n - i as f64).abs() < f64::EPSILON;
                }
            }
            (RuntimeValue::State(s), RuntimeValue::Text(t))
            | (RuntimeValue::Text(t), RuntimeValue::State(s)) => {
                let clean = t.trim()
                    .trim_matches(['\'', '"', '[', ']', '{', '}', '<', '>'])
                    .trim();
                if clean == "0" && *s == QuadState::Await {
                    return true;
                }
                if (clean == "1" || clean == "+1")
                    && *s == QuadState::Proceed
                {
                    return true;
                }
                if clean == "-1" && *s == QuadState::Rollback {
                    return true;
                }
                if clean.eq_ignore_ascii_case("proceed")
                    || clean.eq_ignore_ascii_case("yes")
                {
                    return *s == QuadState::Proceed;
                }
                if clean.eq_ignore_ascii_case("await")
                    || clean.eq_ignore_ascii_case("ok")
                    || clean.eq_ignore_ascii_case("pause")
                {
                    return *s == QuadState::Await;
                }
                if clean.eq_ignore_ascii_case("rollback")
                    || clean.eq_ignore_ascii_case("no")
                {
                    return *s == QuadState::Rollback;
                }
                if clean.eq_ignore_ascii_case("halt")
                    || clean.eq_ignore_ascii_case("null")
                    || clean.eq_ignore_ascii_case("void")
                {
```

```
                    return *s == QuadState::Halt;
                }
            }
            (RuntimeValue::Number(n), RuntimeValue::Text(t))
            | (RuntimeValue::Text(t), RuntimeValue::Number(n)) => {
                let clean = t.trim()
                    .trim_matches(['\'', '"', '[', ']', '{', '}', '<', '>'])
                    .trim();
                if let Ok(parsed) = clean.parse::<f64>() {
                    return (*n - parsed).abs() < f64::EPSILON;
                }
            }
            _ => {}
        }

        false
    }

    // ---------------------------------------------------------------------------
    // Universal Capacity Metric
    // ---------------------------------------------------------------------------

    /// Measures the physical memory capacity of a value in units of 5-trit cells.
    ///
    /// Fulfills capacity bound tests like: `{ Character' <= [ 1 Cell ] }`.
    pub fn capacity_in_cells(&self) -> usize {
        match self {
            RuntimeValue::Character { .. } => 1,
            RuntimeValue::Cell(_) => 1,
            RuntimeValue::State(_) => 1,
            RuntimeValue::Number(_) => 1,
            RuntimeValue::Text(t) => t.len(),
            RuntimeValue::Vessel(v) => v.capacity,
            RuntimeValue::EntityRecord { values, .. } => values.len(),
            RuntimeValue::Vector(v) => v.iter().map(|e| e.capacity_in_cells()).sum(),
        }
    }

    // ---------------------------------------------------------------------------
    // Terminal ANSI Color Formatting
    // ---------------------------------------------------------------------------

    /// Formats the runtime value with ANSI color styling for terminal output.
    pub fn format_colored(&self) -> String {
        match self {
            RuntimeValue::State(QuadState::Proceed) => "\x1b[1;32m+1:PROCEED\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Await) => "\x1b[1;33m0:AWAIT\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Rollback) => "\x1b[1;31m-1:ROLLBACK\x1b[0m".to_string(),
            RuntimeValue::State(QuadState::Halt) => "\x1b[1;35mNULL:HALT\x1b[0m".to_string(),
            RuntimeValue::Number(n) => format!("\x1b[1;36m{n}\x1b[0m"),
            RuntimeValue::Text(t) => format!("\x1b[0;32m\"{t}\"\x1b[0m"),
            RuntimeValue::Cell(c) => format!("\x1b[1;35m{c}\x1b[0m"),
            RuntimeValue::Character { glyph, cell } => {
                let ch = if glyph.is_ascii_graphic() || *glyph == b' ' {
                    (*glyph as char).to_string()
                } else {
                    format!("\\x{:02X}", glyph)
                };
                format!("\x1b[1;33m'{}' <{}>\x1b[0m", ch, cell)
            }
            RuntimeValue::Vessel(v) => format!("\x1b[1;34m{v}\x1b[0m"),
            RuntimeValue::EntityRecord { name, is_immutable, values } => {
                let kind = if *is_immutable { "Axiom" } else { "Entity" };
                format!("\x1b[1;34m{kind} < {name} > [ {} fields ]\x1b[0m", values.len())
            }
            RuntimeValue::Vector(elements) => {
                let formatted: Vec<String> = elements.iter().map(|e| e.format_colored()).collect();
                format!("[{}]", formatted.join(", "))
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Display Trait Implementation & Unit Tests )
// ============================================================================

```

```rust
impl fmt::Display for RuntimeValue {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            RuntimeValue::State(s) => write!(f, "{s}"),
            RuntimeValue::Number(n) => write!(f, "{n}"),
            RuntimeValue::Text(t) => write!(f, "\"{t}\""),
            RuntimeValue::Cell(c) => write!(f, "{c}"),
            RuntimeValue::Character { glyph, .. } => {
                let ch = if glyph.is_ascii_graphic() || *glyph == b' ' {
                    (*glyph as char).to_string()
                } else {
                    format!("\\x{:02X}", glyph)
                };
                write!(f, "'{ch}'")
            }
            RuntimeValue::Vessel(v) => write!(f, "{v}"),
            RuntimeValue::EntityRecord { name, is_immutable, values } => {
                let kind = if *is_immutable { "Axiom" } else { "Entity" };
                write!(f, "{kind} < {name} > [ {} fields ]", values.len())
            }
            RuntimeValue::Vector(elements) => {
                let formatted: Vec<String> = elements.iter().map(|e| e.to_string()).collect();
                write!(f, "[{}]", formatted.join(", "))
            }
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_universal_scalar_unwrappers() {
        let val_num = RuntimeValue::from_number(42.0);
        assert_eq!(val_num.as_number(), Some(42.0));
        assert_eq!(val_num.as_state(), None);

        let val_state = RuntimeValue::from_state(QuadState::Proceed);
        assert_eq!(val_state.as_state(), Some(QuadState::Proceed));
        assert!(val_state.is_proceed());
        assert!(!val_state.is_rollback());

        let val_text = RuntimeValue::from_text("Hello");
        assert_eq!(val_text.as_text(), Some("Hello"));
        assert_eq!(val_text.to_quad_state(), QuadState::Proceed);

        let empty_text = RuntimeValue::from_text("");
        assert_eq!(empty_text.to_quad_state(), QuadState::Await);
    }

    #[test]
    fn test_universal_glyph_and_cell_equality() {
        let ch = RuntimeValue::from_glyph(b'A');
        let text_a = RuntimeValue::from_text("A");
        let num_65 = RuntimeValue::from_number(65.0);

        assert!(ch.is_glyph_equal(&text_a));
        assert!(ch.is_glyph_equal(&num_65));
        assert!(text_a.is_glyph_equal(&num_65));

        let ch_b = RuntimeValue::from_glyph(b'B');
        assert!(!ch.is_glyph_equal(&ch_b));
    }

    #[test]
    fn test_universal_capacity_metric() {
        let ch = RuntimeValue::from_char('X');
        assert_eq!(ch.capacity_in_cells(), 1);

        let cell = RuntimeValue::from_cell(Cell::equilibrium());
        assert_eq!(cell.capacity_in_cells(), 1);

        let text = RuntimeValue::from_text("12345");
        assert_eq!(text.capacity_in_cells(), 5);
    }

    #[test]
```

```
    fn test_literal_operator_and_boundary_casting_duality() {
        let state_await = RuntimeValue::from_state(QuadState::Await);
        let num_zero = RuntimeValue::from_number(0.0);
        let lit_zero = RuntimeValue::from_text("\"0\"");
        let plain_zero = RuntimeValue::from_text("0");
        let cast_bracket = RuntimeValue::from_text("[ 0 ]");
        let cast_brace = RuntimeValue::from_text("{ 0 }");
        let cast_chevron = RuntimeValue::from_text("< 0 >");
        let word_await_single = RuntimeValue::from_text("'AWAIT'");
        let word_await_double = RuntimeValue::from_text("\"AWAIT\"");

        // 1. Literal operator "" clarifies zero as literal value
        assert!(num_zero.is_glyph_equal(&lit_zero));
        assert!(num_zero.is_glyph_equal(&plain_zero));
        assert!(lit_zero.is_glyph_equal(&plain_zero));

        // 2. Boundary casts [ 0 ], { 0 }, < 0 > match zero and Await state
        assert!(num_zero.is_glyph_equal(&cast_bracket));
        assert!(num_zero.is_glyph_equal(&cast_brace));
        assert!(num_zero.is_glyph_equal(&cast_chevron));
        assert!(state_await.is_glyph_equal(&cast_bracket));
        assert!(state_await.is_glyph_equal(&cast_brace));
        assert!(state_await.is_glyph_equal(&cast_chevron));

        // 3. 'AWAIT' keyword-as-word matches state Await and word "AWAIT"
        assert!(state_await.is_glyph_equal(&word_await_single));
        assert!(state_await.is_glyph_equal(&word_await_double));
        assert!(word_await_single.is_glyph_equal(&word_await_double));

        // 4. Positive and Negative Unities Duality (+1: Proceed, -1: Rollback)
        let state_proceed = RuntimeValue::from_state(QuadState::Proceed);
        let num_one = RuntimeValue::from_number(1.0);
        let cast_one = RuntimeValue::from_text("[ +1 ]");
        let word_proceed = RuntimeValue::from_text("'PROCEED'");
        assert!(num_one.is_glyph_equal(&cast_one));
        assert!(state_proceed.is_glyph_equal(&cast_one));
        assert!(state_proceed.is_glyph_equal(&word_proceed));

        let state_rollback = RuntimeValue::from_state(QuadState::Rollback);
        let num_neg_one = RuntimeValue::from_number(-1.0);
        let cast_neg_one = RuntimeValue::from_text("[ -1 ]");
        let word_rollback = RuntimeValue::from_text("'ROLLBACK'");
        assert!(num_neg_one.is_glyph_equal(&cast_neg_one));
        assert!(state_rollback.is_glyph_equal(&cast_neg_one));
        assert!(state_rollback.is_glyph_equal(&word_rollback));

        // 5. NULL / Void Duality
        let state_halt = RuntimeValue::from_state(QuadState::Halt);
        let cast_null = RuntimeValue::from_text("[ NULL ]");
        let word_null = RuntimeValue::from_text("'NULL'");
        assert!(state_halt.is_glyph_equal(&cast_null));
        assert!(state_halt.is_glyph_equal(&word_null));
    }
}
```